

Contents

Programsko inženjerstvo ak.god 2025./2026.	3
Sveučilište u Zagrebu	3
Fakultet elektrotehnike i računarstva	3
Dvoranko	3
Opis projektnog zadatka	3
Cilj projektnog zadatka	4
Problematika projektnog zadatka	4
Postojeća slična rješenja	4
Opseg projektnog zadatka	4
Moguće nadogradnje	6
Funkcionalni zahtjevi	7
Nefunkcionalni zahtjevi	8
Zahtjevi za performanse	8
Zahtjevi za pouzdanost i dostupnost	8
Zahtjevi za upotrebljivost	8
Zahtjevi za kompatibilnost	9
Zahtjevi za sigurnost	9
Dionici	9
Aktori i njihovi funkcionalni zahtjevi	9
Obrasci uporabe	10
Dijagrami obrazaca uporabe	10
1. Visokorazinski dijagram obrazaca uporabe cijelog sustava	10
2. dijagram obrazaca uporabe za ključne značajke	11
3. dijagram obrazaca uporabe za korisničke uloge	12
3.1 Neprijavljeni korisnik	12
3.2 Prijavljeni korisnik	12
3.3 Vlasnik dvorane (iznajmljivač)	13
3.4 Administrator	13
3.5 Sustav e-pošte (automatizirani akter)	13
Opis obrazaca uporabe	13
UC1 – Registracija korisnika	13
UC2 – Prijava korisnika	13
UC3 – Pregled dostupnih dvorana	14
UC4 – Pregled slobodnih termina	14
UC5 – Rezervacija termina	14
UC6 – Pregled javnih događanja	14
UC7 – Pregled vlastitih rezervacija	15
UC8 – Zahtjev za promjenom uloge	15
UC9 – Slanje zahtjeva za objavu dvorane	15
UC10 – Upravljanje vlastitim dvoranama	15
UC11 – Pregled zahtjeva za rezervacije	15
UC12 – Prihvatanje / odbijanje rezervacija	16
UC13 – Primanje e-mail obavijesti	16
UC14 – Pregled zahtjeva za ulogu vlasnika	16
UC15 – Pregled zahtjeva za objavu dvorana	16
UC16 – Nadzor nad korisnicima i dvoranama	16

UC17 – Obavješćavanje korisnika putem e-maila	17
Sekvencijski dijagrami	17
Provjera uključenosti ključnih funkcionalnosti u obrasce uporabe	19
Arhitektura sustava	20
Opis arhitekture	20
Stil arhitekture	20
Podsustavi	20
Preslikavanje na radnu platformu	20
Spremišta podataka	20
Mrežni protokoli	20
Globalni upravljački tok	20
Sklopovskoprogramski zahtjevi	20
Obrazloženje odabira arhitekture	21
Izbor arhitekture temeljen na principima oblikovanja	21
Razmatrane alternative	21
Organizacija sustava na visokoj razini	21
Klijent-poslužitelj	21
Baza podataka	21
Grafičko sučelje	21
Organizacija aplikacije	23
Frontend i Backend slojevi	23
Baza podataka	23
Opis tablica	23
Users	23
Dvorana	24
Kategorija	25
Termin	25
Adresa	25
Mjesto	26
Zahtjev_iznajmljivac	26
Zahtjev_oglas	26
Slika_dvorana	27
Dijagram baze podataka	27
Dijagram razreda	28
Cijela Web-Aplikacija	28
Modeli	30
Kontroleri	31
DTO	32
Servisi	33
Sigurnost i pomoćne klase	34
Dinamičko ponašanje aplikacije	34
UML dijagrami stanja	34
UML dijagrami aktivnosti	35
Dijagram komponenata	35
Dijagram razmještaja	35
Ispitivanje komponenti	36
Ispitivanje sustava	36
Korištene tehnologije i alati	38
Opis prisutpa aplikaciji na javnom poslužitelju	40
Dnevnik sastajanja	41

Plan rada	41
Tablica aktivnosti	41
Dijagram pregleda promjena	42
Ključni izazovi i rješenja	42

Programsko inženjerstvo ak.god 2025./2026.

Sveučilište u Zagrebu

Fakultet elektrotehnike i računarstva

Dvoranko

Tim: TG10.1

Ime tima : LeProgi

Članovi: Vid Martin Lulić, Dino Maras, Borna Navratil, Eva Huljaj, Matija Vandek, Ivan Vučak

Nastavnik: Vlado Sruk

Opis projektnog zadatka

U urbanim sredinama poput Zagreba, sve je veća potreba za pronalaženjem i rezervacijom prostora za različite namjene: radionice, konferencije, proslave ili sportske aktivnosti. Trenutačno korisnici najčešće pretražuju i kontaktiraju dvorane ručno putem interneta, društvenih mreža, telefona ili e-maila, što je dugotrajan i često neorganiziran proces. Upravo iz te potrebe proizašla je naša ideja Dvoranko web-aplikacije, čiji je cilj omogućiti jednostavniju, bržu i pregledniju rezervaciju dvorana u Zagrebu na jednom mjestu. Dvoranko okuplja ponudu različitih prostora u jedinstven sustav, čime olakšava komunikaciju između iznajmljivača i korisnika te modernizira proces rezervacije i oglašavanja dvorana.

Aplikaciju Dvoranko moguće je koristiti kao običan korisnik, vlasnik dvorane ili administrator. Kada korisnik prvotno otvori aplikaciju nazivamo ga neprijavljeni korisnik. On tada ima pristup početnoj stranici aplikacije, gdje mu se prikazuju objavljene dvorane, i oglasnoj ploči te se mora prijaviti kako bi postao običan (prijavljeni) korisnik. Obični korisnici imaju mogućnost pretraživanja i rezervacije dvorana. Tijekom rezervacije korisnik mora definirati namjenu prostora ako je iznajmljivač to zatražio, broj gostiju te označiti hoće li događanje biti javno ili privatno. U slučaju javnog događanja, ono postaje vidljivo svim korisnicima unutar aplikacije, čime se potiče sudjelovanje zajednice i promocija događanja u gradu. Prijavljenim korisnicima je omogućeno i upravljanje vlastitim rezervacijama. Mogu ih pregledavati, mijenjati ili otkazati prema pravilima sustava.

Vlasnici dvorana imaju dodatne funkcionalnosti koje im omogućuju upravljanje svojim prostorima. Nakon registracije, obični korisnik može zatražiti nadogradnju uloge u vlasnika dvorane. Kada administrator odobri zahtjev, vlasnik može dodavati nove dvorane, unositi njihove opise, veličine, kapacitete i fotografije te određivati cijene i dostupne termine. Sustav automatski obavještava vlasnika putem e-maila kada korisnik pošalje zahtjev za rezervaciju termina. Vlasnik može pregledati sve zahtjeve, prihvatiti ih ili odbiti, te tako jednostavno upravljati svojim poslovanjem putem aplikacije.

Administrator ima najvišu razinu pristupa i odgovoran je za održavanje sigurnosti unutar sustava. Njegove funkcionalnosti uključuju odobravanje zahtjeva korisnika za objavu dvorana te potvrdu promjene korisničke uloge u ulogu iznajmljivača dvorane ili prostora. Također, administrator prati rad aplikacije, provjerava točnost unesenih podataka te ima uvid u sve rezervacije i korisničke aktivnosti kako bi osigurali pouzdanost i transparentnost sustava.

Svi korisnici aplikaciji pristupaju putem prijave koristeći e-mail adresu, a autentifikacija se odvija uz pomoć OAuth2 protokola radi zaštite osobnih podataka. Aplikacija je u potpunosti responzivna i prilagođena korištenju na različitim uređajima, računalima i mobilnim telefonima, čime se osigurava jednostavan pristup i preglednost.

Kroz ovakav sustav, Dvoranko omogućuje digitalizaciju i unaprjeđenje procesa rezervacije prostora u Zagrebu. Korisnici mogu pronaći dvoranu koja im odgovara u svega nekoliko klikova, dok vlasnici mogu učinkovito upravljati svojim prostorima i terminima bez potrebe za dodatnim alatima. Administratori osiguravaju da cijeli sustav funkcionira pouzdano i sigurno.

Na taj način, Dvoranko stvara povezan “ekosustav” u kojem su svi sudionici uključeni u transparentan i učinkovit proces rezervacije dvorana.

Cilj projektnog zadatka

Osnovni cilj ovog projektnog zadatka je omogućiti korisnicima jednostavno pretraživanje i rezervaciju prostora za različite svrhe poput sastanaka, radionica, sportskih aktivnosti i sajмова, a vlasnicima prostora objavljivanje i iznajmljivanje svojih prostora. Dvoranko omogućuje centralizirano mjesto za sve ponude dvorana u Zagrebu. Ideja je napraviti pristupačno sučelje za sve korisnike kako bi se prije navedeni proces olakšao i modernizirao.

Dvoranko web-aplikacija pruža korisnicima brzo i efikasno rezerviranje termina i organiziranje javnih ili privatnih događanja. S druge strane pruža vlasnicima prostora mogućnost oglašavanja svojih prostora bez potrebe za izradom vlastitih stranica i sustava oglašavanja. Konačni cilj je stvoriti centraliziranu platformu koja će objediniti ponudu javnih i privatnih dvorana u Zagrebu, olakšati korisnicima pronalazak slobodnih termina te povećati iskorištenost dostupnih prostora.

Problematika projektnog zadatka

Trenutačno u Zagrebu ne postoji jedinstveno digitalno rješenje koje omogućuje jednostavno pretraživanje i rezervaciju dvorana ili prostora. Dosadašnji načini rezervacija dvorana iznimno ovise o tome u čijem su vlasništvu i koliko se te osobe samostalno promoviraju. Najčešći način rezerviranja ovakvih prostora je putem mobilnih telefona ili putem e-maila. Iako su ove metode jednostavne i poznate većini ljudi, takvim pristupom se cjelokupni proces znatno usporava. Korisnici koji traže prostor često moraju kontaktirati više lokacija zasebno što troši i njihovo vrijeme, a i vrijeme voditelja prostora koji moraju odgovarati na upite, te povećava vjerojatnost pogreške pri rezervacijama. Naša aplikacija nudi način ubrzanja ovog procesa gdje bi korisnici na jednom mjestu imali pregled slobodnih termina dvorana, čiji su ih vlasnici oglasili u sustavu.

Još jedan od problema na koji bi naša aplikacija utjecala je promoviranje javnih događanja. Brojni sajmovi ili predstave se svakodnevno održavaju u Zagrebu za koje veliki broj ljudi ne zna. Ako korisnik rezervira prostor u svrhe javnog događanja, može rezervaciju označiti upravo kao javno događanje te ga oglasiti na oglasnoj ploči aplikacije. Ostali korisnici će imati pregled javnih događanja koja se odvijaju u gradu te će se moći prijaviti na ono što ih zanima.

Postojeća slična rješenja

Postoji mnogo aplikacija i web stranica koje nude rješenja za ovaj problem. Većina tih rješenja se fokusira na rezervaciju termina za sportska događanja poput turnira i treninga. U tom aspektu se naša aplikacija razlikuje od već postojećih.

Jedan od primjera sličnog rješenja je Sport Router. Sport Router je mobilna aplikacija koja korisnicima pruža intuitivno i jednostavno iskustvo rezerviranja sportskih terena za vlastite potrebe. Termine je moguće pretraživati po vrsti sporta, željenom vremenu te lokaciji terena ili dvorane. Kada korisnik instalira aplikaciju može isključivo rezervirati termin. Za objavu vlastitog centra potrebno je ispuniti prijavu na njihovoj web stranici (<https://sportrouter.com/#connectYourCenter>).

Korisnicima se nakon prijave otvara jednostavno korisničko sučelje s izbornikom sportova i izbornikom datuma u tekućem tjednu. Kada korisnik odabere sport i datum, ispod toga mu se prikazuju slobodni termini na različitim lokacijama (slika 1.1). Na slici je prikazano korisničko sučelje za mobilne aplikacije.

Odabirom određene dvorane vidljivi su podaci o samom prostoru. Vidljiva je lokacija prostora, radno vrijeme te opis prostora koji vlasnik napiše kada objavljuje svoj prostor u aplikaciji (slika 1.2). Vidljivi su termini po danima u tjednu te raspoređeni po satima.

Naša aplikacija ima nekoliko sličnih funkcionalnosti i uporaba kao i Sport Router. Neke od njih su: mogućnost prijave kao korisnik i promjene te uloge u iznajmljivača, mogućnost opisivanja prostora koji korisnik objavljuje, pregled objavljenih prostora. Razliku pronalazimo u namjeni prostora ili razlogu rezervacije. Dvoranko omogućuje rezervaciju dvorana ili prostora za javna događanja poput sajмова, predstava uz mogućnost rezervacije dvorana za sportske aktivnosti.

Opseg projektnog zadatka

Projekt Dvoranko obuhvaća izradu web-aplikacije koja omogućuje centralizirano i jednostavno upravljanje procesom pretraživanja, rezervacije i oglašavanja dvorana u Zagrebu. Sustav je zamišljen kao trodijelna platforma koja povezuje korisnike, vlasnike dvorana i administratore u jedinstveno digitalno okruženje.

Funkcionalni opseg



Figure 1: Slika 1

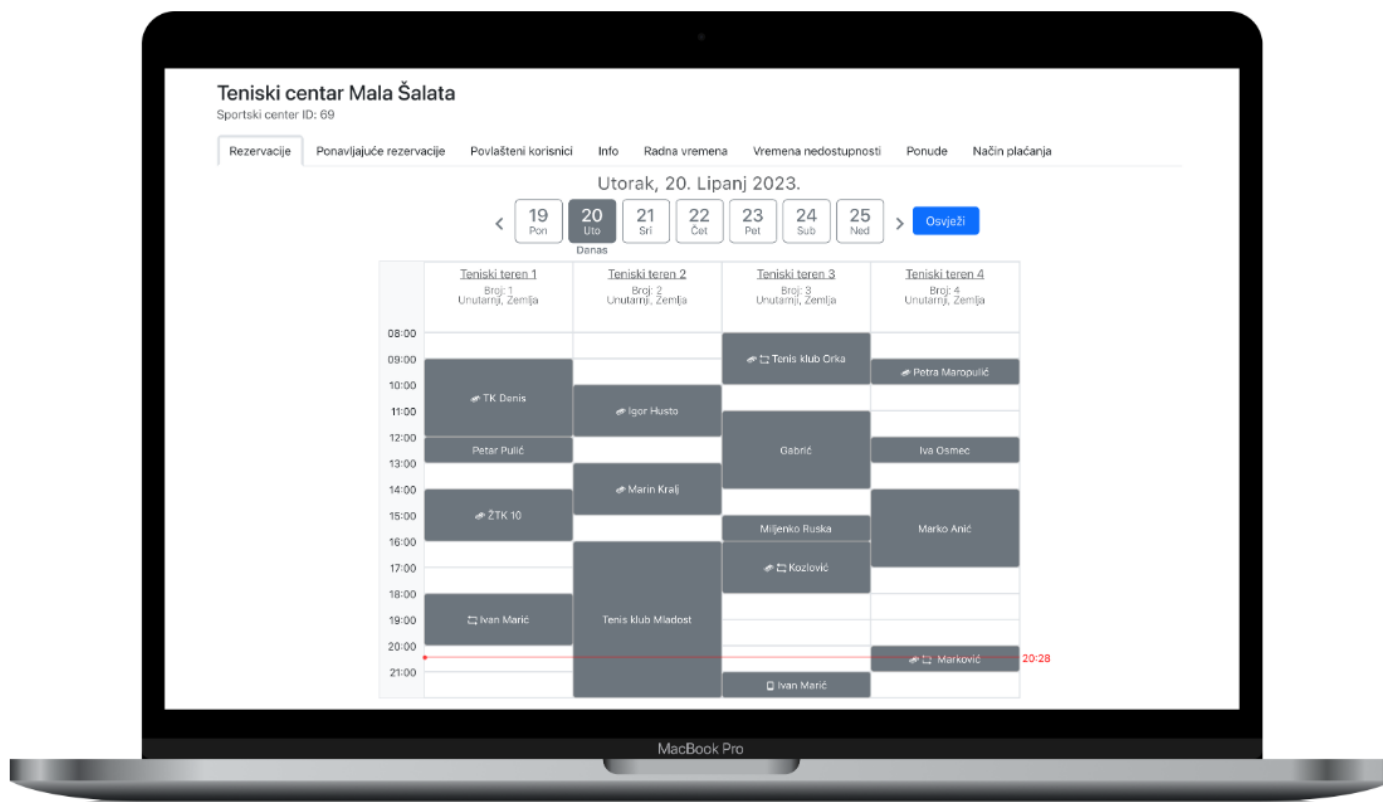


Figure 2: Slika 2

1. Neprijavljeni korisnici > - Pregled javno dostupnih dvorana i oglasne ploče. > - Pregled prikaza dvorana na karti > - Registracija i prijava u sustav.
2. Prijavljeni korisnici (obični korisnici) > - Detaljno pretraživanje i filtriranje dostupnih dvorana prema različitim parametrima (cijena, veličina, dostupnost termina, namjena prostora). > - Pregled informacija o dvoranama (opis, fotografije, dostupni termini). > - Rezervacija dvorane uz unos potrebnih informacija (broj gostiju, namjena, javno/privatno događanje). > - Upravljanje vlastitim rezervacijama – pregled, izmjena i otkazivanje. > - Zahtjev za promjenu korisničke uloge iz običnog korisnika u vlasnika dvorane.
3. Vlasnici dvorana > - Unos i upravljanje vlastitim prostorima (dodavanje, uređivanje i brisanje dvorana). > - Definiranje detalja o prostoru (opis, kapacitet, veličina, cijena, fotografije, dostupni termini). > - Pregled zaprimljenih zahtjeva za rezervacije i upravljanje njima (odobranje ili odbijanje zahtjeva). > - Primanje automatskih obavijesti e-mailom o novim zahtjevima za rezervaciju.
4. Administratori > - Odobranje zahtjeva korisnika za promjenu uloge u vlasnika dvorane. > - Pregled i verifikacija unesenih podataka o dvoranama. > - Upravljanje oglasima i održavanje integriteta sustava. > - Uvid u sve rezervacije i korisničke aktivnosti radi kontrole i sigurnosti sustava.

Tehnički opseg > - Platforma: Web-aplikacija dostupna putem internetskog preglednika. > - Autentifikacija: Prijava putem e-mail adrese uz korištenje OAuth2 protokola radi sigurnosti korisničkih podataka. > - Responzivni dizajn: Prilagođen prikaz i funkcionalnosti za rad na računalima, tabletima i mobilnim uređajima. > - Baza podataka: Centralizirano spremište podataka o korisnicima, dvoranama, rezervacijama i javnim događanjima. > - E-mail notifikacije: Automatsko slanje obavijesti korisnicima i vlasnicima o relevantnim aktivnostima u sustavu (potvrda registracije, zahtjevi za rezervaciju, odobrenja uloga itd.).

Moguće nadogradnje

Dvoranko je osmišljen kao fleksibilna platforma koja je lako prilagodljiva različitim potrebama korisnika. Naša trenutačna aplikacija se koncentrira na problem rezervacija dvorana u Zagrebu. Jedna od mogućih prilagodba je proširiti područje na druge gradove poput Rijeke, Splita ili Osijeka. Moguće se proširiti i na druge vrste prostora poput kazališta, koncertnih

dvorana, konferencijskih sala i slično. Omogućili bismo rezervaciju kazališta za predstave ili konferencijskih sali za domjenke čime bi se baza klijenata još više povećala.

Druga potencijalna nadogradnja je izrada mobilne aplikacije za Android ili iOS platforme, što bi korisnicima omogućilo brži i pristupačniji način rezervacije dvorana. Mobilna verzija mogla bi sadržavati push notifikacije za nove ponude dvorana ili novo dostupne termine. Daljnja nadogradnja može uključivati dodatne funkcionalnosti. Može se integrirati sustav ocjenjivanja prostora koji bi korisnicima omogućio davanje povratne informacije o prostoru ili usluzi. Time bi se povećala transparentnost među korisnicima te bi vlasnici dvorana ili prostora dobili uvid u kvalitetu prostora koje nude. Mogao bi se integrirati i prikaz statistike vlasnicima prostora koji uključuje broj novih klijenata, ukupan broj rezervacija i slično.

Funkcionalni zahtjevi

ID zahtjeva	Opis	Prioritet	Kriterij prihvaćanja
F-01	Sustav omogućuje korisnicima prijavu putem Google računa koristeći OAuth2 autentifikaciju.	Visok zahtjev dionika	Korisnik koji se prvi put prijavi putem Google računa automatski se registrira i pohranjuje u bazu podataka.
F-02	Sustav podržava tri korisničke uloge: obični korisnik, vlasnik dvorane i administrator.	Visok postojeći sustav	Novi korisnici dobivaju ulogu "običnog korisnika". Administrator i vlasnici dvorana imaju dodatne ovlasti definirane u sustavu.
F-03	Sustav omogućuje neprijavljenim korisnicima pregled javnog popisa dvorana (osnovni podaci: naziv, lokacija, kapacitet)	Visok zahtjev dionika	Neprijavljeni korisnici mogu pregledavati popis dvorana, ali ne mogu otvoriti detalje niti rezervirati dvoranu.
F-04	Sustav omogućuje neprijavljenim korisnicima pregled javnih događanja na oglasnoj ploči.	Visok zahtjev dionika	Neprijavljeni korisnici mogu pregledavati oglasnu ploču.
F-05	Sustav prikazuje sve objavljene dvorane na interaktivnoj karti s osnovnim informacijama dostupnim svim korisnicima.	Srednji zahtjev dionika	Na karti se prikazuju označene sve objavljene dvorane s osnovnim podacima.
F-06	Samo prijavljeni korisnici mogu pregledati detaljne informacije o dvorani (opis, slike, cijene, dostupni termini).	Visok zahtjev dionika	Pokušaj neprijavljenog korisnika da otvori detalje preusmjerava ga na stranicu za prijavu.
F-07	Sustav omogućuje prijavljenim korisnicima rezervaciju slobodnih termina određene dvorane.	Visok zahtjev dionika	Prijavljeni korisnik može odabrati slobodan termin i rezervirati ga.
F-08	Sustav prilikom rezervacije zahtijeva unos svrhe događanja, tipa događanja (javno/privatno) i broja gostiju.	Visok postojeći sustav	Sustav ne dopušta završetak rezervacije dok svi obavezni podatci nisu uneseni.
F-09	Sustav omogućuje korisniku pregled, izmjenu i otkazivanje vlastitih rezervacija.	Srednji dokument zahtjeva	Korisnik na profilu može pregledati i upravljati vlastitim rezervacijama.
F-10	Korisnik može zatražiti promjenu uloge u vlasnika dvorane (iznajmljivača).	Visok dokument zahtjeva	Korisnik ispunjava zahtjev za promjenu uloge koji administrator mora potvrditi ili odbiti.
F-11	Vlasnik dvorane može oglasiti svoju dvoranu putem aplikacije.	Visok dokument zahtjeva	Vlasnik unosi podatke o dvorani (opis, slike, kapacitet, cijenu, lokaciju) i šalje zahtjev administratoru.
F-12	Svaka objavljena dvorana mora sadržavati sliku, opis i lokaciju.	Srednji postojeći sustav	Sustav ne dopušta objavu dvorane bez svih obaveznih informacija.
F-13	Svaki termin mora sadržavati datum, vrijeme i cijenu najma.	Visok postojeći sustav	Sustav ne dopušta objavu termina bez unesenih podataka o datumu, vremenu i cijeni.
F-14	Administrator može odobriti ili odbiti zahtjeve za objavu novih dvorana.	Visok povratne informacije korisnika	Administrator odlučuje o zahtjevu, a sustav automatski šalje obavijest vlasniku dvorane.

ID zahtjeva	Opis	Prioritet	Kriterij prihvaćanja
F-15	Administrator može odobriti ili odbiti zahtjeve korisnika za promjenu uloge u vlasnika dvorane.	Visokopovratne informacije korisnika	Sustav obavještava korisnika o odluci administratora elektroničkom poštom.
F-16	Sustav automatski šalje e-mail obavijest korisniku o statusu zahtjeva za rezervaciju (odobreno/odbijeno).	Srednjopovratne informacije korisnika	Nakon odluke vlasnika, korisnik automatski prima obavijest putem e-pošte.
F-17	Sustav automatski šalje e-mail obavijest vlasniku dvorane o novom zahtjevu za rezervaciju.	Srednjopovratne informacije korisnika	Kada korisnik rezervira termin, vlasnik automatski prima e-mail s informacijama o zahtjevu.
F-18	Sustav omogućuje vlasniku uređivanje informacija o svojim objavljenim dvoranama.	Srednjopostojeći sustav	Vlasnik može mijenjati opis, slike, cijenu i termine dvorane, a izmjene se spremaju u bazu.
F-19	Sustav omogućuje korisnicima filtriranje i pretraživanje dvorana po lokaciji, kapacitetu i cijeni.	Srednji zahtjev korisnika	Korisnik može primijeniti filtere za pretragu dvorana prema zadanim parametrima.
F-20	Sustav omogućuje administratoru pregled svih korisnika, dvorana i rezervacija te njihovo uređivanje ili brisanje.	Visokodokument zahtjeva	Administrator ima pristup administracijskoj ploči i može mijenjati ili uklanjati zapise u sustavu.

Nefunkcionalni zahtjevi

Zahtjevi za performanse

ID zahtjeva	Opis	Prioritet
NF-1.1	Web-aplikacija mora u potpunosti učitati sve elemente glavnih stranica unutar 2-3 sekunde (Home page, login page).	Visok

Zahtjevi za pouzdanost i dostupnost

ID zahtjeva	Opis	Prioritet
NF-2.1	Aplikacija je dostupna 24 sata dnevno tokom cijele godine osim za vrijeme održavanja ("downtime")	Visok
NF-2.2	Sustav mora osigurati da podatci o prijavama ne budu izgubljeni ili oštećeni (sve uspješno obavljene rezervacije moraju biti trajno spremljene u bazi podataka)	Visok

Zahtjevi za upotrebljivost

ID zahtjeva	Opis	Prioritet
NF-3.1	Mogućnost povećavanja i smanjivanja fonta	Manji
NF-3.2	Aplikacija mora biti jednostavna za korištenje, s jasnim navigacijskim elementima za korisnike svih razina tehničke pismenosti.	Srednji

Zahtjevi za kompatibilnost

ID		
zahtjeva	Opis	Prioritet
NF-4.1	Aplikacija mora raditi ispravno na najnovijim verzijama svih glavnih preglednika (Chrome, Firefox, Safari, Edge).	Visok
NF-4.2	Sučelje aplikacije treba biti responzivnog dizajna te podržavati rad na mobilnim uređajima unutar web preglednika	Srednji

Zahtjevi za sigurnost

ID		
zahtjeva	Opis	Prioritet
NF-5.1	Sustav koristi OAuth2 za prijavu i upravljanje sesijama kako bi osigurao siguran pristup korisničkim podacima.	Visok
NF-5.2	Osobni podaci korisnika (e-mail, lozinke, povijest rezervacija,...) moraju biti pohranjeni u skladu s važećim propisima (GDPR).	Visok
NF-5.3	Pristup podacima i funkcionalnostima mora biti ograničen na temelju korisničkih uloga	Visok

Dionici

D1 -> neprijavljeni korisnik - krajnji korisnici koji imaju pristup početnoj stranici web-aplikacije, oglasnoj ploči i karti te se za daljnje korištenje moraju registrirati (ulogirati)

D2 -> prijavljeni korisnik - korisnici koji rezerviraju termine za željene dvorane, prijavljuju se na javna događanja te koriste ostale funkcionalnosti aplikacije

D3 -> iznajmljivači - imaju sve mogućnosti kao obični korisnici uz dodatne opcije da oglašavaju svoje dvorane, upravljaju informacijama o dvorani, potvrđuju ili odbijaju termine

D4 -> administrator - osoba koja održava sustav te upravlja zahtjevima korisnika

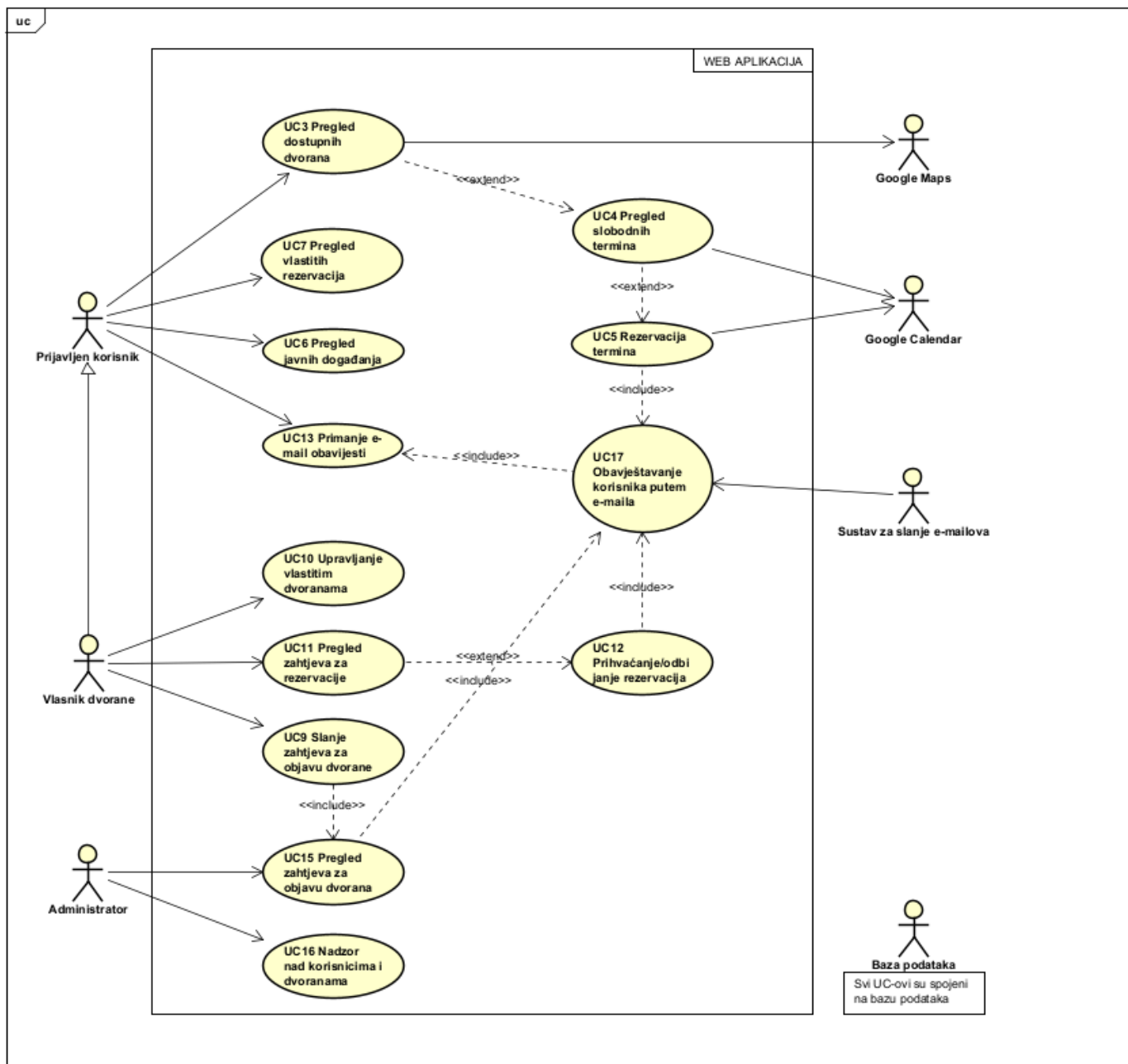
Aktori i njihovi funkcionalni zahtjevi

- A1 - Neprijavljeni korisnik (inicijator) > - Pregled početne stranice aplikacije (Home page) (FZ-03) > - Pregled popisa dvorana, oglasne ploče te karte s označenim dvoranama (FZ-03, FZ-04, FZ-05) > - Mogućnost registracije putem e-maila (FZ-01)
- A2 - Prijavljeni korisnik (inicijator) > - Registracija i prijava putem e-maila (FZ-01) > - Pregled dostupnih dvorana i slobodnih termina (FZ-06, FZ-19) > - Rezervacija termina (FZ-07) > - Definiranje namjene rezervacije (javno/privatno, broj gostiju) (FZ-08) > - Pregled i prijava na javna događanja > - Pregled i upravljanje vlastitim rezervacijama (FZ-09) > - Zahtjev za promjenom uloge u vlasnika dvorane (FZ-10)
- A3 - Vlasnik dvorane / Iznajmljivač (inicijator/sudionik) > - Sve mogućnosti običnog korisnika (FZ-01, FZ-06, FZ-07, FZ-08, FZ-09, FZ-10, FZ-19) > - Slanje zahtjeva za objavu dvorane (unesi opis, veličinu, kapacitet, cijenu, slike) (FZ-11, FZ-12) > - Upravljanje vlastitim oglašenim dvoranama (FZ-18) > - Pregled zahtjeva za rezervacije svojih dvorana (FZ-17) > - Prihvaćanje ili odbijanje rezervacija (FZ-16) > - Primanje e-mail obavijesti o novim zahtjevima (FZ-17)
- A4 - Administrator (sudionik) > - Pregled i odobravanje zahtjeva za: > - Promjenu uloge korisnika u vlasnika dvorane (FZ-15) > - Objavu nove dvorane (FZ-14) > - Nadzor nad sustavom (uvid u korisnike, rezervacije, dvorane) (FZ-20) > - Održavanje sigurnosti i pouzdanosti aplikacije (FZ-20)

Obrasci uporabe

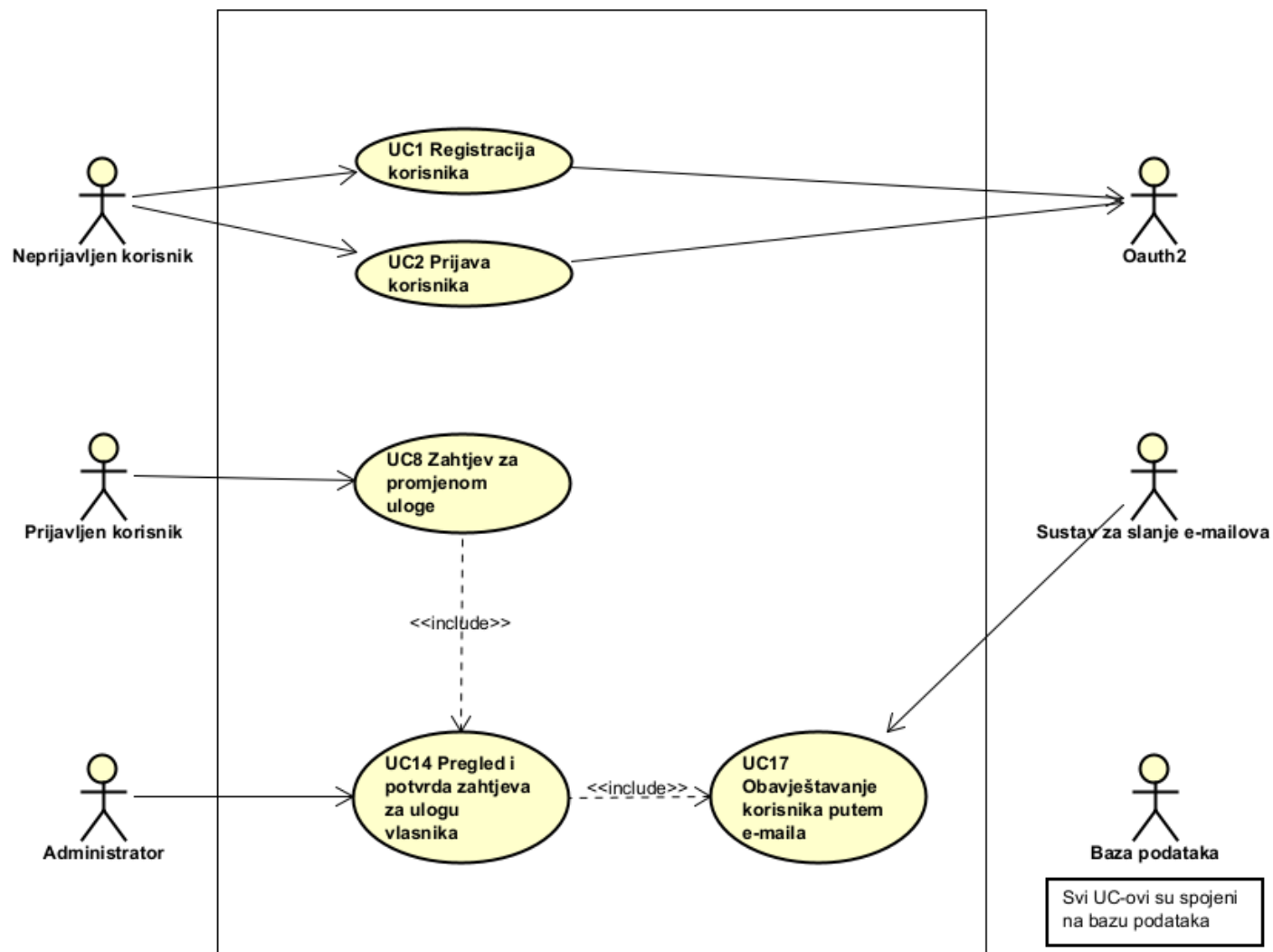
Dijagrami obrazaca uporabe

1. Visokorazinski dijagram obrazaca uporabe cijelog sustava

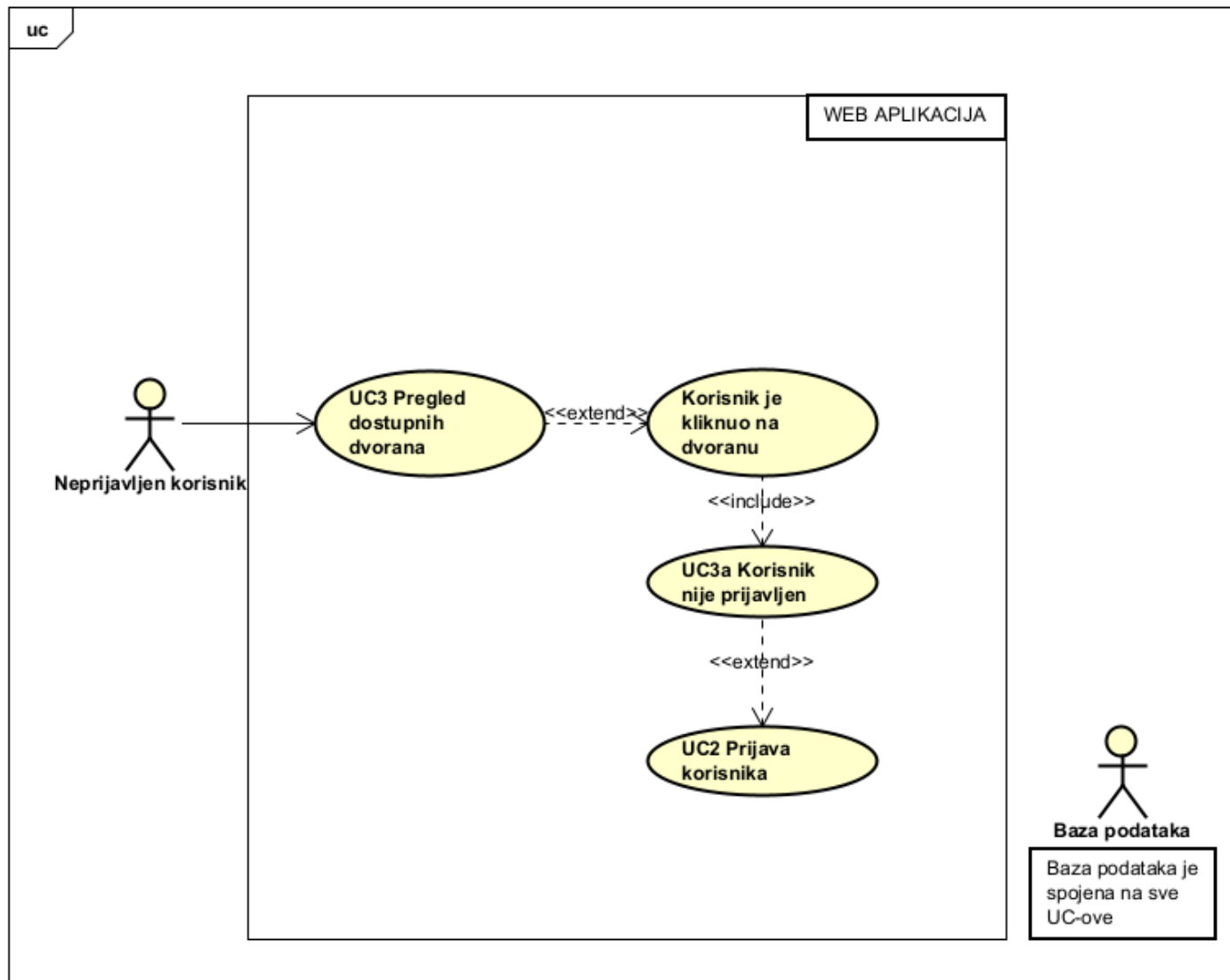


3.1 slika - Prikaz glavnih funkcionalnosti sustava Dijagram prikazuje dodatne pasivne sustave, glavne aktore i njihove glavne funkcionalnosti tijekom korištenja aplikacije te je obuhvaćena većina slučajeva korištenja.

2. dijagram obrazaca uporabe za ključne značajke



3.2 slika - Prikaz prijave i proces promjene uloge korisnika u iznajmljivača dvorane Dijagram prikazuje prijavu korisnika i proces promocije korisnika u iznajmljivača dvorane. Neprijavljen korisnik se registrira ukoliko se prvi put prijavljuje u aplikaciju te se mora pohraniti novi korisnik.



3.3 slika - Prikaz pregledavanja dvorana ukoliko korisnik nije prijavljen Dijagram prikazuje mogućnost neprijavljenog korisnika da pregledava dostupne dvorane, no ukoliko bi pokušao rezervirati dvoranu ili vidjeti slobodne termine bio bi prusmjeren na prijavu.

3. dijagram obrazaca uporabe za korisničke uloge

3.1 Neprijavljeni korisnik

ID	Use Case naziv	Kratak opis
UC1	Registracija korisnika	Korisnik se registrira pomoću Google računa
UC2	Prijava korisnika	Korisnik se prijavljuje pomoću Google računa
UC3	Pregled dostupnih dvorana	Prikaz svih objavljenih dvorana s terminima

3.2 Prijavljeni korisnik

ID	Use Case naziv	Kratak opis
UC3	Pregled dostupnih dvorana	Prikaz svih objavljenih dvorana s terminima
UC4	Pregled slobodnih termina	Pregled slobodnih termina za odabranu dvoranu
UC5	Rezervacija termina	Rezervacija termina, unos svrhe, tipa događaja i broja gostiju
UC6	Pregled javnih događanja	Pregled javnih događanja objavljenih na oglasnoj ploči

ID	Use Case naziv	Kratak opis
UC7	Pregled vlastitih rezervacija	Pregled, izmjena ili otkazivanje vlastitih rezervacija
UC8	Zahtjev za promjenom uloge	Slanje zahtjeva za ulogu vlasnika dvorane
UC13	Primanje e-mail obavijesti	Sustav šalje e-mail obavijesti korisniku

3.3 Vlasnik dvorane (iznajmljivač)

ID	Use Case naziv	Kratak opis
UC9	Slanje zahtjeva za objavu dvorane	Slanje podataka o dvorani na odobravanje administratoru
UC10	Upravljanje vlastitim dvoranama	Uređivanje informacija o objavljenim dvoranama
UC11	Pregled zahtjeva za rezervacije	Pregled korisničkih zahtjeva za rezervaciju
UC12	Prihvaćanje/odbijanje rezervacija	Odluka o prihvaćanju ili odbijanju zahtjeva za rezervaciju

3.4 Administrator

ID	Use Case naziv	Kratak opis
UC14	Pregled zahtjeva za ulogu vlasnika	Pregled i odlučivanje o zahtjevima korisnika za ulogom vlasnika dvorane
UC15	Pregled zahtjeva za objavu dvorana	Odobrovanje ili odbijanje prijave novih dvorana
UC16	Nadzor nad korisnicima i dvoranama	Pregled korisnika, dvorana i sustavnih aktivnosti

3.5 Sustav e-pošte (automatizirani akter)

ID	Use Case naziv	Kratak opis
UC17	Obavješćavanje korisnika putem e-maila	Automatsko slanje e-mail obavijesti korisnicima

Opis obrazaca uporabe

UC1 – Registracija korisnika

Glavni sudionik: Neprijavljeni korisnik

Cilj: Prva prijava u aplikaciju pomoću Google accounta

Sudionici: Baza podataka, OAuth2

Preduvjet: Korisnik se nikada prije nije prijavio s tim Google računom

- Osnovni tijek:**
1. Korisnik otvara stranicu za prijavu
 2. Klikne gumb za Google prijavu
 3. Nakon autentifikacije, vraća se u aplikaciju prijavljen
 4. Sustav sprema korisnika u bazu podataka

UC2 – Prijava korisnika

Glavni sudionik: Neprijavljeni korisnik

Cilj: Prijava u aplikaciju pomoću Google accounta

Sudionici: Baza podataka, OAuth2

Preduvjet: Korisnik je već ranije registriran

- Osnovni tijek:** 1. Korisnik otvara stranicu za prijavu
2. Klikne Google prijavu
3. Nakon autentifikacije vraća se u aplikaciju
4. Sustav učitava podatke povezane s korisnikom
-

UC3 – Pregled dostupnih dvorana

Glavni sudionik: Korisnik

Cilj: Pregled svih objavljenih dvorana

Sudionici: Baza podataka, Google Maps API **Preduvjet:** Korisnik pregledava početnu stranicu web-aplikacije

- Osnovni tijek:** 1. Korisnik otvara popis dvorana
2. Sustav dohvaća i prikazuje dvorane (opis, slike, cijena)
3. Korisnik može filtrirati rezultate (lokacija, veličina, cijena)
4. Korisnik pritisne na karticu “Karta” i prikažu mu se sve dvorane na karti pomoću servisa Google Maps API

Odstupanja: - 2a. Nema dostupnih dvorana → poruka „Trenutno nema dostupnih dvorana.”

- 3a. Ukoliko korisnik nije prijavljen, sustav ga preusmjerava na stranicu za login —

UC4 – Pregled slobodnih termina

Glavni sudionik: Prijavljen Korisnik

Cilj: Pregledati slobodne termine određene dvorane

Sudionici: Baza podataka, Google Calendar API **Preduvjet:** Dvorana je objavljena i ima upisan raspored termina te korisnik mora biti prijavljen

- Osnovni tijek:** 1. Korisnik odabire dvoranu
2. Sustav dohvaća slobodne termine i prikazuje ih pomoću servisa Google Calendar API 3. Prikazuju se datumi, sati i cijene
-

UC5 – Rezervacija termina

Glavni sudionik: Prijavljen Korisnik

Cilj: Rezervirati željeni termin

Sudionici: Vlasnik dvorane, Sustav e-pošte, Baza podataka

Preduvjet: Korisnik je prijavljen i termin je slobodan

- Osnovni tijek:** 1. Korisnik odabire slobodan termin
2. Ispunjava formu (svrha, broj gostiju, tip događaja)
3. Potvrđuje rezervaciju
4. Sustav sprema rezervaciju u bazu
5. Sustav šalje e-mail vlasniku

Odstupanja: - 4a. Termin zauzet → „Termin više nije dostupan.”

UC6 – Pregled javnih događanja

Glavni sudionik: Prijavljen Korisnik

Cilj: Vidjeti javna događanja drugih korisnika

Sudionici: Baza podataka

Preduvjet: Postoje objavljena javna događanja

- Osnovni tijek:** 1. Korisnik otvara oglasnu ploču
2. Sustav prikazuje sva javna događanja
-

UC7 – Pregled vlastitih rezervacija

Glavni sudionik: Prijavljen Korisnik

Cilj: Vidjeti, izmijeniti ili otkazati vlastite rezervacije

Sudionici: Baza podataka

Preduvjet: Korisnik je prijavljen u sustav

Osnovni tijek: 1. Korisnik otvara svoj profil
2. Sustav prikazuje sve njegove rezervacije

Odstupanja: - 4a. Nema rezerviranih termina → „Nije rezerviran niti jedan termin”

UC8 – Zahtjev za promjenom uloge

Glavni sudionik: Prijavljen Korisnik

Cilj: Zatražiti status vlasnika dvorane

Sudionici: Administrator, Baza podataka

Preduvjet: Korisnik je prijavljen kao običan korisnik

Osnovni tijek: 1. Korisnik ispunjava zahtjev
2. Sustav sprema i prosljeđuje zahtjev administratoru
3. Administrator odlučuje (UC14)

UC9 – Slanje zahtjeva za objavu dvorane

Glavni sudionik: Vlasnik dvorane

Cilj: Dodati novu dvoranu u sustav

Sudionici: Administrator, Baza podataka

Preduvjet: Korisnik ima ulogu vlasnika dvorane

Osnovni tijek: 1. Vlasnik unosi podatke (opis, kapacitet, slike, cijena)
2. Sustav sprema i šalje zahtjev administratoru
3. Administrator odobrava/odbija (UC15)

UC10 – Upravljanje vlastitim dvoranama

Glavni sudionik: Vlasnik dvorane

Cilj: Uređivanje svojih objavljenih dvorana

Sudionici: Baza podataka

Preduvjet: Dvorana je već objavljena

Osnovni tijek: 1. Vlasnik otvara listu svojih dvorana
2. Odabire dvoranu
3. Mijenja informacije
4. Sustav sprema izmjene

UC11 – Pregled zahtjeva za rezervacije

Glavni sudionik: Vlasnik dvorane

Cilj: Pregledati sve zaprimljene zahtjeve za svoje dvorane

Sudionici: Baza podataka

Preduvjet: Postoje zahtjevi

Osnovni tijek: 1. Sustav prikazuje sve zahtjeve
2. Vlasnik bira zahtjev
3. Nastavlja s UC12 (odbij ili prihvati)

UC12 – Prihvaćanje / odbijanje rezervacija

Glavni sudionik: Vlasnik dvorane

Cilj: Donijeti odluku o rezervaciji

Sudionici: Sustav e-pošte, Baza podataka

Preduvjet: Postoji zahtjev

Osnovni tijek: 1. Vlasnik odabire zahtjev

2. Odluka se bilježi u bazu

3. Sustav šalje e-mail korisniku

UC13 – Primanje e-mail obavijesti

Glavni sudionik: Korisnik

Cilj: Primanje obavijesti o promjenama/rezervacijama

Preduvjet: Obavijest je poslana

Osnovni tijek:

- Korisnik prima e-mail obavijest

UC14 – Pregled zahtjeva za ulogu vlasnika

Glavni sudionik: Administrator

Cilj: Odobriti ili odbiti zahtjeve za promjenu uloge

Sudionici: Baza podataka

Preduvjet: Postoje zahtjevi

Osnovni tijek: 1. Administrator pregleda zahtjev

2. Odobri ili odbije

3. Sustav šalje e-mail korisniku

UC15 – Pregled zahtjeva za objavu dvorana

Glavni sudionik: Administrator

Cilj: Odluka o objavi novih dvorana

Sudionici: Baza podataka

Preduvjet: Zaprimljeni zahtjevi vlasnika

Osnovni tijek: 1. Administrator pregleda zahtjeve

2. Donosi odluku

3. Sustav šalje obavijest vlasniku

UC16 – Nadzor nad korisnicima i dvoranama

Glavni sudionik: Administrator

Cilj: Nadgledati sustav, dvorane, korisnike

Sudionici: Baza podataka

Preduvjet: Administrator je prijavljen

Osnovni tijek: 1. Otvara administracijsku ploču

2. Pregledava korisnike, dvorane, rezervacije

3. Može reagirati (brisanje, blokiranje itd.)

UC17 – Obavješćavanje korisnika putem e-maila

Glavni sudionik: Sustav e-pošte

Cilj: Automatski slati obavijesti korisnicima

Sudionici: Baza podataka

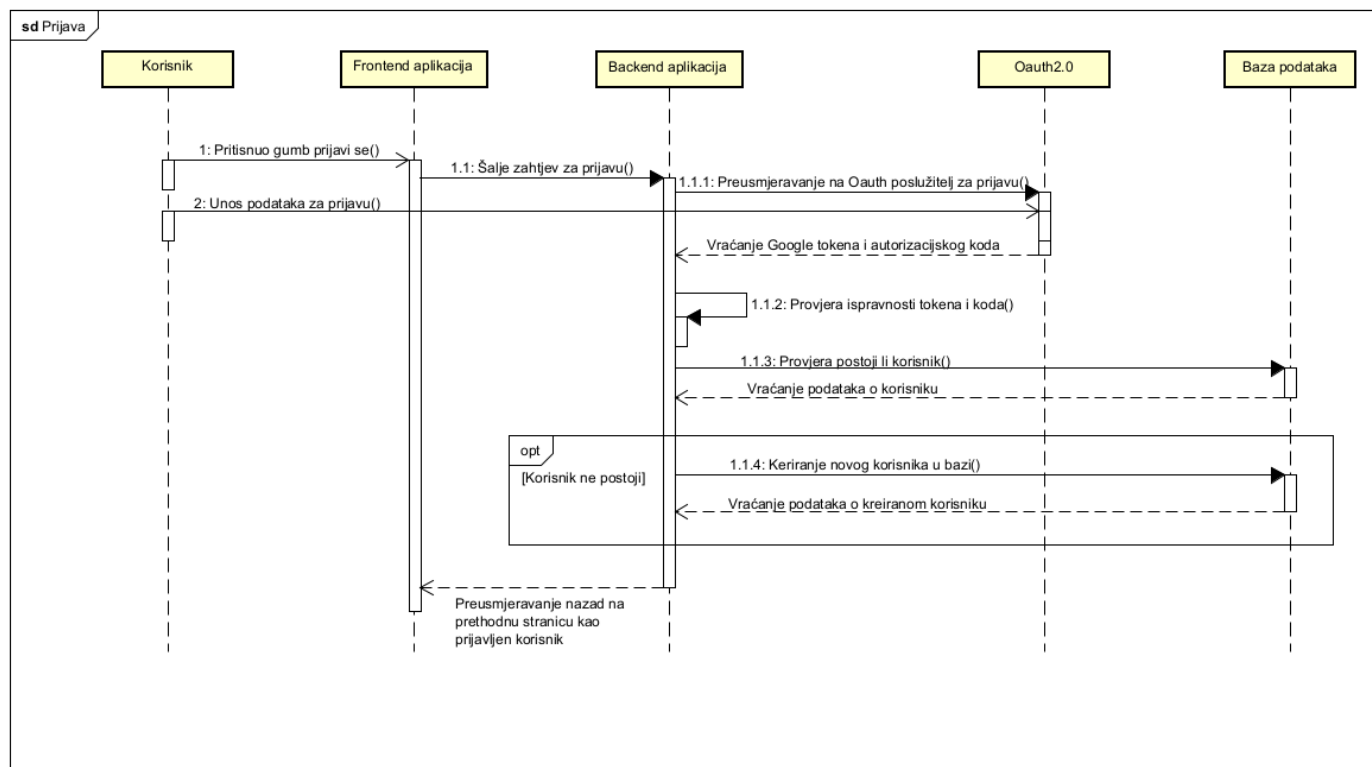
Preduvjet: Nastao je događaj koji zahtijeva e-mail obavijest

Osnovni tijek: 1. Sustav detektira potrebnu obavijest

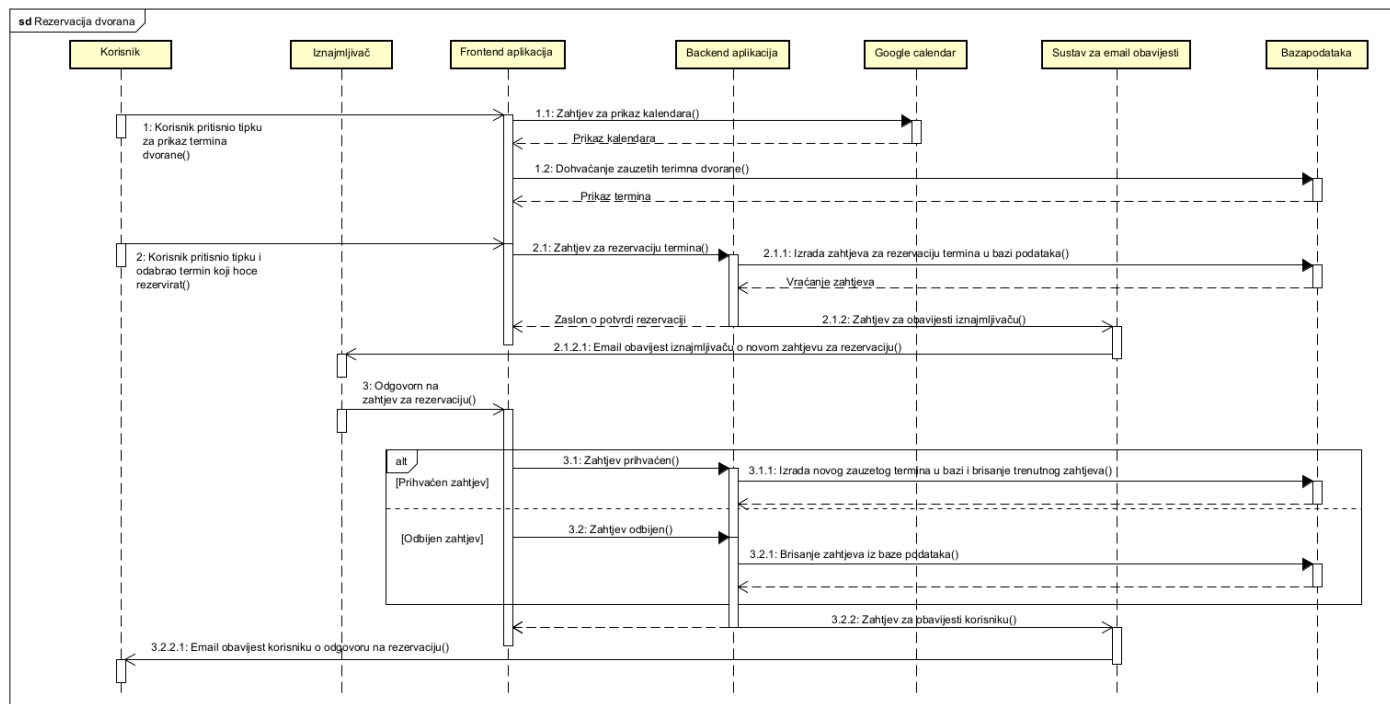
2. Generira e-mail

3. Automatski šalje korisniku

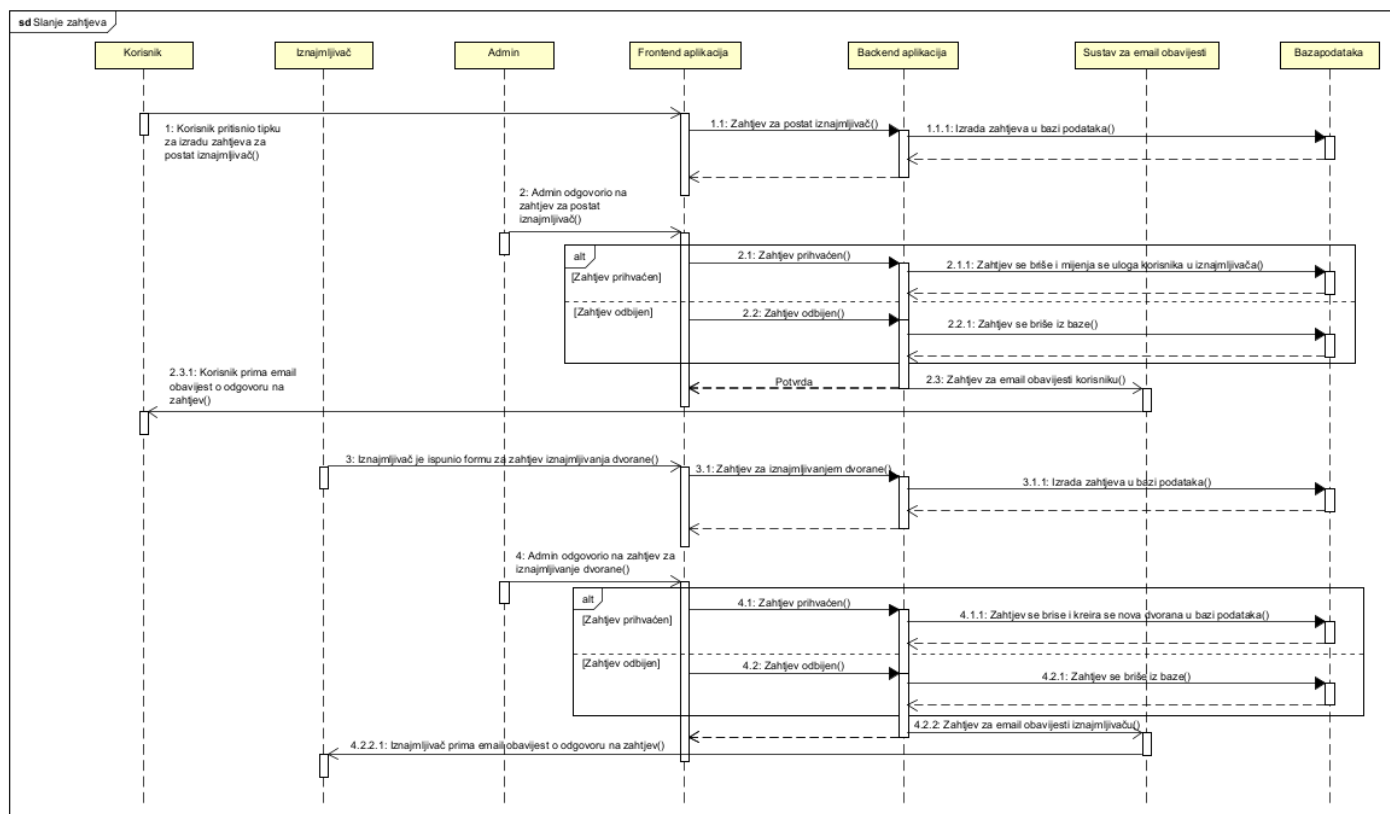
Sekvencijski dijagrami



Slika 3.4 - Sekvencijski dijagra za prijavu korisnika Sekvencijski dijagram prikazuje postupak prijave korisnika putem Google OAuth2 autentifikacije. Korisnik pritisne gumb za prijavu, a frontend aplikacija šalje zahtjev backendu koji preusmjerava korisnika na OAuth poslužitelj. Nakon uspješne prijave, poslužitelj vraća token i autorizacijski kod backendu koji provjerava njihovu ispravnost te traži korisnika u bazi. Ako korisnik postoji, vraćaju se njegovi podaci, a ako ne, kreira se novi zapis u bazi. Na kraju korisnik je preusmjeren natrag u aplikaciju kao prijavljen.



Slika 3.5 - Sekvencijski dijagra za prikaz funkcionalnosti rezervacije dvorane Sekvencijski dijagram „Rezervacija dvorana“ prikazuje proces rezervacije termina između korisnika, iznajmljivača i sustava. Korisnik putem aplikacije prvo pregledava slobodne termine koje backend dohvaća iz baze podataka i Google kalendara. Nakon što odabere željeni termin, frontend šalje zahtjev backendu, koji pohranjuje zahtjev o rezervaciji i obavještava iznajmljivača putem emaila. Iznajmljivač zatim odlučuje hoće li zahtjev prihvatiti ili odbiti. U slučaju prihvaćanja, backend kreira zauzeti termin u bazu, ako je odbijen, zahtjev se briše. Na kraju, sustav putem email obavijesti javlja korisniku odgovor na zahtjev rezervacije. Proces jasno prikazuje međusobnu komunikaciju i automatizirano upravljanje terminima.



Slika 3.6 - Sekvencijski dijagram za prikaz funkcionalnosti slanja zahtjeva administratoru i odgovaranja na zahtjeve Sekvencijski dijagram „Slanje zahtjeva“ prikazuje proces u kojem korisnik i iznajmljivač šalju zahtjeve administratoru,

a sustav obrađuje i obavještava sudionike o ishodu. Korisnik prvo šalje zahtjev da postane iznajmljivač; frontend ga prosljeđuje backendu koji ga sprema u bazu podataka. Administrator zatim odlučuje hoće li zahtjev prihvatiti ili odbiti. Ako je prihvaćen, korisniku se mijenja uloga u iznajmljivača, a ako nije, zahtjev se briše. Nakon toga sustav šalje korisniku email obavijest o odluci. Nakon što postane iznajmljivač, korisnik može poslati zahtjev za iznajmljivanje dvorane. Proces je sličan — zahtjev se sprema u bazu, administrator ga odobrava ili odbija, ukoliko je odobren se kreira nova dvorana za iznajmljivanje, a ako nije samo se zahtjev briše. Sustav na kraju automatski šalje iznajmljivaču email obavijest s odgovorom. Dijagram jasno prikazuje automatiziran i transparentan tijek obrade zahtjeva i obavještavanja.

Provjera uključenosti ključnih funkcionalnosti u obrasce uporabe

ID obrasca uporabe	Naziv obrasca uporabe	Povezani funkcionalni zahtjevi	Obrazloženje poveznice
UC1	Registracija korisnika	F-01, F-02	Registracija putem Google računa (OAuth2) i inicijalno dodjeljivanje uloge običnog korisnika.
UC2	Prijava korisnika	F-01, F-02	Prijava putem Google računa i prepoznavanje uloge korisnika.
UC3	Pregled dostupnih dvorana	F-03, F-05, F-06, F-19	Prikaz popisa dvorana, prikaz na karti, ograničen pristup neprijavljenih korisnika i mogućnost filtriranja.
UC4	Pregled slobodnih termina	F-06, F-13	Pregled detalja dvorane i prikaz dostupnih termina s cijenama.
UC5	Rezervacija termina	F-07, F-08, F-13, F-17	Rezervacija termina, unos potrebnih podataka, provjera dostupnosti i automatsko obavještavanje vlasnika putem e-maila.
UC6	Pregled javnih događanja	F-04	Prikaz oglasne ploče sa svim javnim događanjima.
UC7	Pregled vlastitih rezervacija	F-09	Pregled, izmjena i otkazivanje vlastitih rezervacija korisnika.
UC8	Zahtjev za promjenom uloge	F-10, F-15	Korisnik može zatražiti promjenu uloge u vlasnika, a administrator odlučuje.
UC9	Slanje zahtjeva za objavu dvorane	F-11, F-12, F-14	Vlasnik unosi podatke o dvorani, sustav ih validira i prosljeđuje administratoru na odobrenje.
UC10	Upravljanje vlastitim dvoranama	F-18, F-12	Vlasnik uređuje informacije o svojim objavljenim dvoranama (opis, slike, cijene, termini).
UC11	Pregled zahtjeva za rezervacije	F-17, F-18	Pregled zaprimljenih zahtjeva za rezervacije povezan je s e-mail obavijestima i uređivanjem statusa zahtjeva.
UC12	Prihvatanje / odbijanje rezervacija	F-17	Sustav šalje e-mail obavijesti korisniku o odluci vlasnika te obavještava obje strane.

ID obrasca uporabe	Naziv obrasca uporabe	Povezani funkcionalni zahtjevi	Obrazloženje poveznice
UC13	Primanje e-mail obavijesti	F-16	Automatsko slanje e-mail obavijesti korisnicima o rezervacijama i promjenama.
UC14	Pregled zahtjeva za ulogu vlasnika	F-15	Administrator odobrava ili odbija zahtjeve za promjenu uloge korisnika.
UC15	Pregled zahtjeva za objavu dvorana	F-14	Administrator odlučuje o objavi dvorana i obavještava vlasnika o odluci.
UC16	Nadzor nad korisnicima i dvoranama	F-20	Administrator nadzire korisnike, dvorane i rezervacije te može uređivati ili brisati zapise.
UC17	Obavješćavanje korisnika putem e-maila	F-16, F-17	Sustav automatski generira i šalje e-mail obavijesti o događajima u sustavu.

Arhitektura sustava

Opis arhitekture

Stil arhitekture

Izabrali smo stil klijent-poslužitelj. Razlozi biranja ovog stila navedeni su u nastavku.

Podsustavi

Podsustavi koji se koriste u aplikaciji su OAuth2, Maps Embed API i Google Calendar API. OAuth2 koristi se za autentifikaciju korisnika i provjeravanje lozinki prilikom login-a. Maps Embed API se koristi za prikazivanje lokacija dvorana na karti. Google Calendar API se koristi za prikaz dostupnosti termina prilikom rezervacije dvorane.

Preslikavanje na radnu platformu

Arhitektura će biti implementirana na platformi Render jer je besplatna i omogućuje nam otvaranje servisa za aplikaciju i servisa za bazu podataka.

Spremišta podataka

Za pohranu podataka koristimo relacijsku bazu podataka koja je detaljnije objašnjena u nastavku.

Mrežni protokoli

Za komunikaciju između komponenata sustava koristiti će se protokol HTTP.

Globalni upravljački tok

Korisničko sučelje za svaku korisničku interakciju sa sustavom šalje upit programskoj potpori sustava. Ona obrađuje taj upit i po potrebi pristupa bazi podataka kako bi dohvatila neke podatke. Nakon obrade, traženi resursi se vraćaju korisničkom sučelju, koje ih tada može prikazati korisniku.

Sklopovskoprogramski zahtjevi

Aplikaciju će biti moguće koristiti bez obzira na operacijski sustav i na više različitih uređaja (stolno računalo, prijenosno računalo, prijenosni telefon...). Sve što je potrebno jest da uređaj na kojem se pokreće aplikacija ima web preglednik koji podržava HTML, CSS i Javascript. Uz to, potrebno je da uređaj ima stabilnu internetsku vezu s *download* i *upload* brzinama od barem 1Mb po sekundi.

Obrazloženje odabira arhitekture

Arhitektura klijent-poslužitelj je najčešće korištena arhitektura kod izrade web aplikacija. Izabrali smo ju jer nudi jasnu podjelu na klijenta i poslužitelja što se slagalo s potrebama našeg sustava.

Izbor arhitekture temeljen na principima oblikovanja

Arhitektura klijent-poslužitelj bila je dobar izbor za naš sustav zbog njene niske povezanosti. Klijent(frontend) i poslužitelj(backend) nisu povezani, neovisni su jedan o drugome. To nam omogućuje razvoj oba dijela sustava u isto vrijeme i olakšava pronalazak izvora greški. Ovaj stil također je fleksibilan i zato nam je lakše dodati novu funkcionalnost u sustav. Još jedna prednost ovog stila je visoka kohezija koja nam je omogućila da podijelimo zadatke na puno podzadataka radi lakšeg razumijevanja, testiranja i ispravljanja grešaka.

Razmatrane alternative

Odmah smo se složili da nam je arhitektura klijent-poslužitelj najbolji izbor za aplikaciju pa nismo razmatrali druge alternative.

Organizacija sustava na visokoj razini

Klijent-poslužitelj

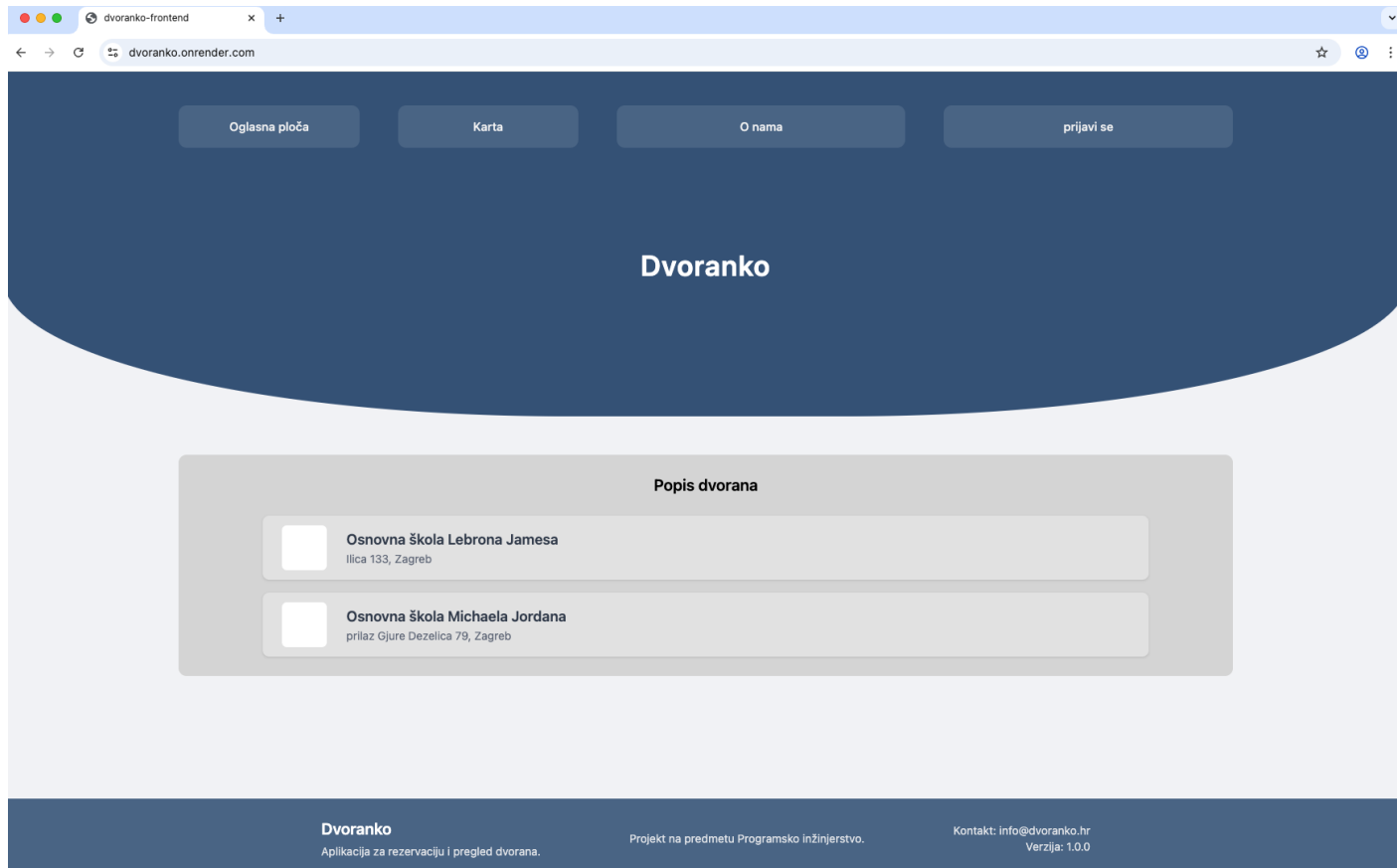
Sustav je organiziran tako da je korisničko sučelje(frontend) klijent, a programska potpora(backend) poslužitelj.

Baza podataka

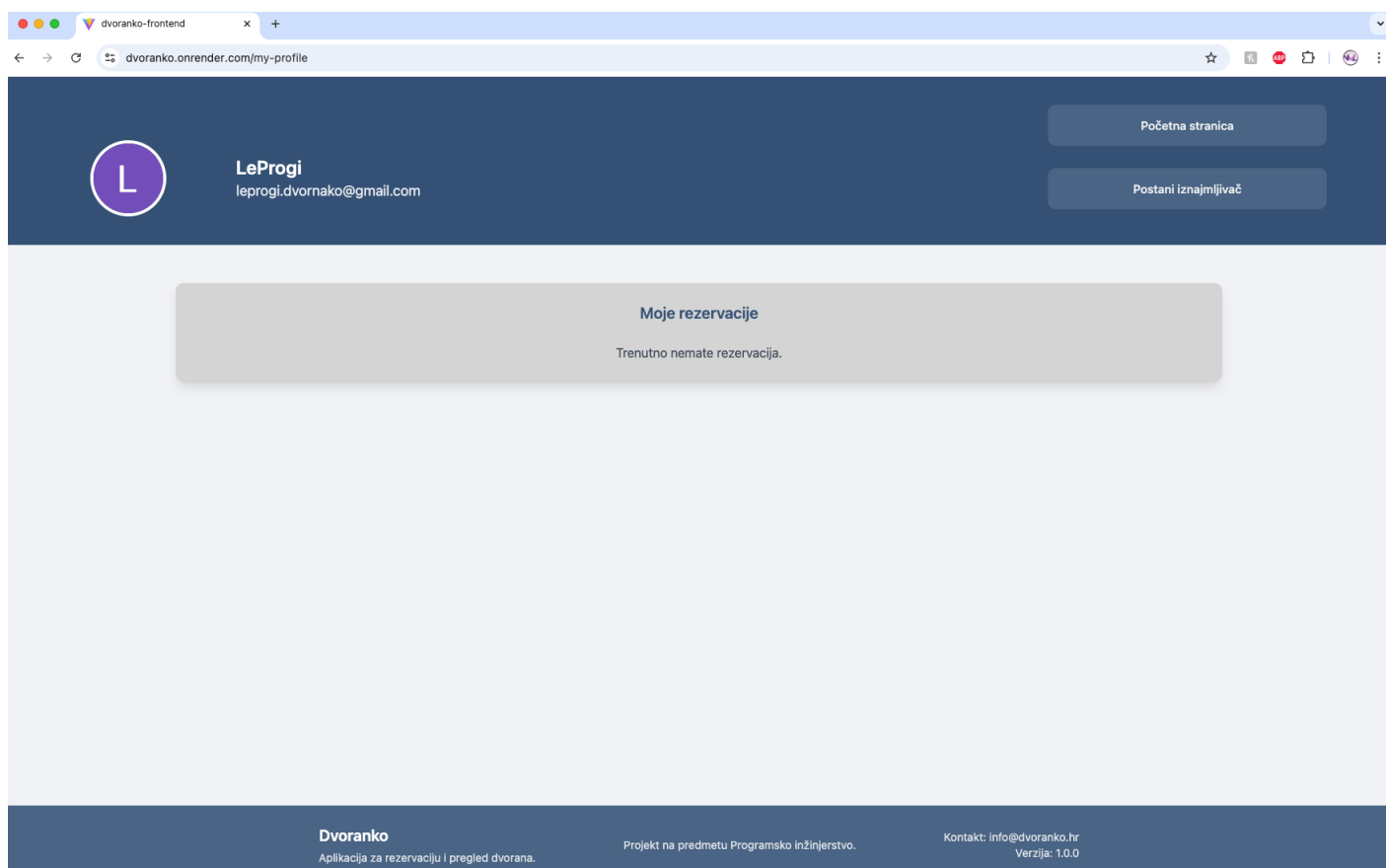
Bazu podataka koristimo za spremanje podataka o svim dvoranama i njihovim lokacijama. Lokacije su spremljene pomoću njihovih koordinata i adresa i povezane s gradom u kojem se nalaze. Uz dvorane vežu se kategorije koje nam govore za što je dvorana namijenjena i njihovi iznajmljivači. Nadalje, u bazu spremamo zauzete termine koje vežemo uz određene dvorane i korisnike koji su ih rezervirali. Svi iznajmljivači i korisnici spremljeni su skupa s adminima u tablici. Također, spremamo podatke o zahtjevima za mijenjanje uloge iz korisnika u iznajmljivača i zahtjeve za oglašavanje novih dvorana.

Grafičko sučelje

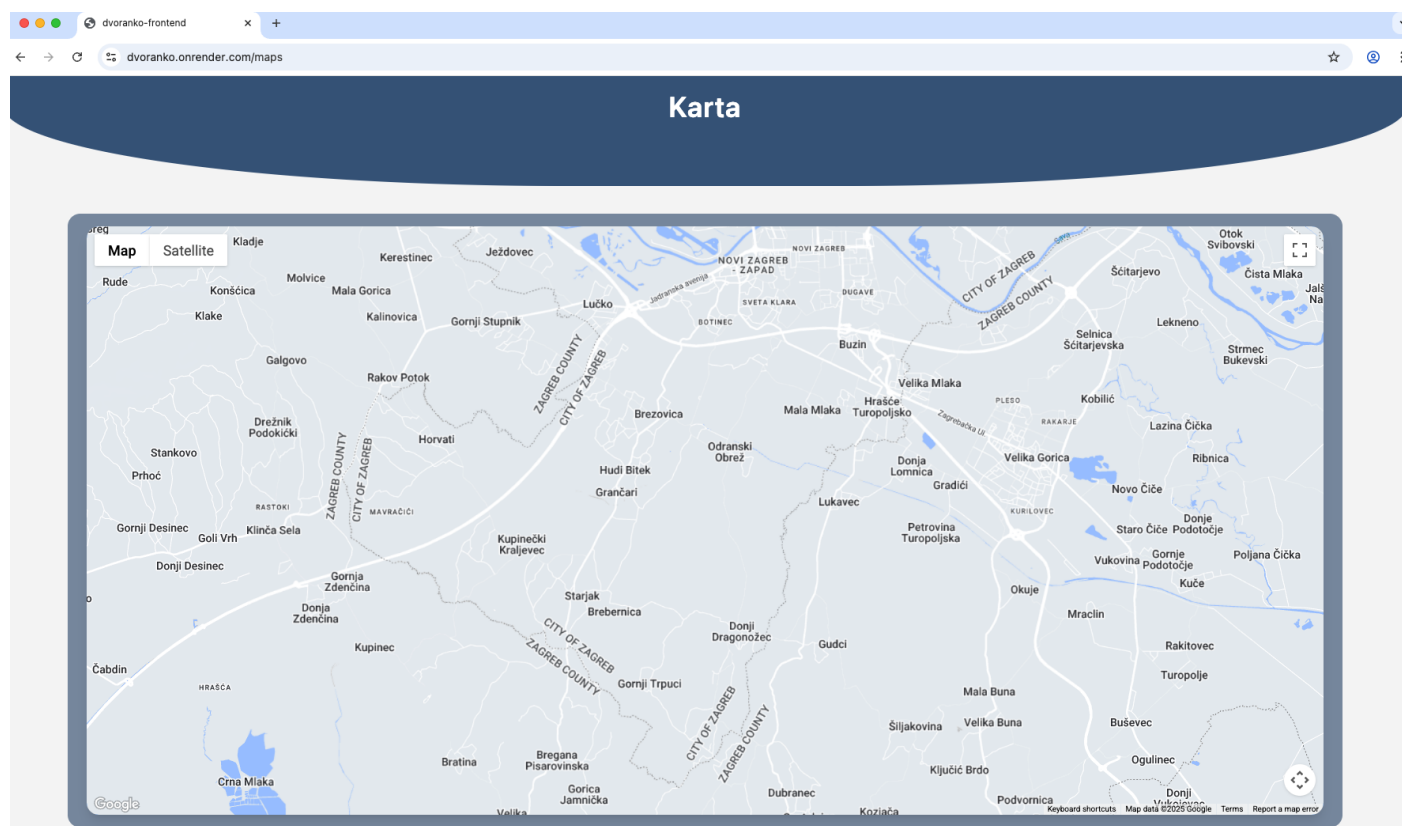
Klijent informacije o dvoranama i korisnicima dobiva zahtjevima koje šalje na poslužitelj. Kada klijent primi odgovor on formatira informacije odgovora te ih adekvatno prikaže na grafičkom sučelju.



Grafičko sučelje je jednostavno organizirano kako bi ga korisnici svih razina tehničke pismenosti mogli razumjeti. S početne stranice je vidljiva lista dostupnih dvorana te gumbi za prijavu korisnika, gumb za prelazak na stranicu oglasne ploče te gumb za prelazak na kartu s označenim dvorana.



Kada je korisnik logiran pritiskom na profilnu sliku sa početne stranice pristupa svom profilu gdje vidi svoje podatke te podatke o svojim rezervacijama.



Pritiskom na gumb “Karta” korisnik vidi prikaz karte grada Zagreba te na njoj označene lokacije dostupnih dvorana.

Organizacija aplikacije

Frontend i Backend slojevi

Aplikacija Dvoranko koristi frontend i backend sloj koji međusobno komuniciraju putem REST API-ja.

Frontend sloj sastoji se od prezentacijskog sloja čija je funkcija upravljanje korisničkim sučeljem i interakcijom korisnika s aplikacijom. Sastoji se od HTML-a, CSS-a i JS frameworka React.

Backend sloj sastoji se od poslovnog sloja i podatkovnog sloja. Poslovni sloj sadrži logiku aplikacije i ona obrađuje svu temeljnu logiku aplikacije, tj. određuje kako će se podaci obrađivati i slati nazad korisniku ili u bazu podataka. Za poslovni sloj korišten je Java framework Spring Boot. Podatkovni sloj zaslužan je za direktnom komunikacijom s bazom podataka. Njegov posao je spremanje i dohvaćanje podataka. Za podatkovni sloj korišten je Spring Data JPA koji komunicira s bazom podataka koja je izrađena u PostgreSQL-u.

Baza podataka

Podatci su spremljeni u relacijskoj bazi podataka PostgreSQL. Ona je implementirana pomoću Render servisa. Aplikacija se na bazu povezuje putem SQL upita. Glavne komponente baze su entiteti Korisnik, Dvorana, Kategorija, Termin, Adresa, Mjesto, ZahtjevOglas i ZahtjevIznajmljivac. Za navedene entitete stvorene su tablice opisane u nastavku. One su spojene preko primarnih i stranih ključeva.

Opis tablica

Users

Sadrži podatke o svim korisnicima, iznajmljivačima i adminima

Atribut	Tip podatka	Opis varijable
id	BIGINT (Primarni ključ)	Jedinstveni identifikator korisnika
name	VARCHAR(255)	Ime korisnika
picture_url	VARCHAR(255)	Poveznica na sliku korisnikovog goolge računa
role	VARCHAR(255)	Uloga korisnika(korisnik, iznajmljivač, admin)
email	VARCHAR(255)	E-mail adresa korisnika
google_id	VARCHAR(255)	Google id korisnikovog google računa

Dvorana

Sadrži podatke o svim dvoranama

Atribut	Tip podatka	Opis varijable
id_dvorana	BIGINT (Primarni ključ)	Jedinstveni identifikator dvorane
naziv_dvorana	VARCHAR(255)	Naziv dvorane
kapacitet	INT	Kapacitet dvorane
opis	VARCHAR(1000)	Opis dvorane(specifična namjena, kontakt iznajmljivača, cijena rezervacije, ...)
termini_tjedan	VARCHAR(70)	Podatci o tome u kojim satima za svaki dan u tjednu iznajmljivač želi iznajmljivati dvoranu
id_adresa	BIGINT (Strani ključ)	Identifikator adrese na kojoj se nalazi dvorana
id_kategorija	BIGINT (Strani ključ)	Identifikator kategorije dvorane
id_vlasnik	BIGINT (Strani ključ)	Identifikator korisnika(iznajmljivača) koji oglašava dvoranu

Kategorija

Sadrži podatke o svim kategorijama(namjenama)

Atribut	Tip podatka	Opis varijable
id_kategorija	BIGINT (Primarni ključ)	Jedinstveni identifikator kategorije
naziv_kategorija	VARCHAR(255)	Naziv kategorije(namjene) dvorane

Termin

Sadrži podatke o svim zauzetim terminima

Atribut	Tip podatka	Opis varijable
datum_vrijeme_start	VARCHAR(8) (Primarni ključ)	Jedinstveni par datuma i vremena početka rezerviranog termina
id_dvorana	BIGINT (Primarni i strani ključ)	Identifikator dvorane na koju se termin odnosi
datum_vrijeme_end	VARCHAR(8)	Datum i vrijeme kraja termina
je_javni_event	INT	Oznaka koja označava je li event privatn ili javan
cijena	INT	Cijena termina
id_korisnik	BIGINT (Strani ključ)	Identifikator korisnika koji je rezervirao termin

Adresa

Sadrži podatke o svim adresama

Atribut	Tip podatka	Opis varijable
id_adresa	BIGINT (Primarni ključ)	Jedinstveni identifikator adrese dvorane
longitude	DOUBLE	Zemljopisna dužina adrese
latitude	DOUBLE	Zemljopisna širina adrese
ulica	VARCHAR(255)	Ulica u kojoj se nalazi dvorana
kucni_broj	INT	Kućni broj na kojem se nalazi dvorana

Atribut	Tip podatka	Opis varijable
id_mjesto	BIGINT (Strani ključ)	Identifikator mjesta u kojem se nalazi dvorana

Mjesto

Sadrži podatke o svim mjestima

Atribut	Tip podatka	Opis varijable
id_mjesto	BIGINT (Primarni ključ)	Jedinstveni identifikator mjesta
postanski_broj	BIGINT	Pošanski broj mjesta
naziv_mjesto	VARCHAR(255)	Naziv mjesta

Zahtjev_iznajmljivac

Sadrži podatke o svim zahtjevima za promjenu uloge u iznajmljivača

Atribut	Tip podatka	Opis varijable
id	INT (Primarni ključ)	Jedinstveni identifikator zahtjeva
user_id	INT (Strani ključ)	Identifikator korisnika koji želi postati iznajmljivač

Zahtjev_oglas

Sadrži podatke o svim zahtjevima za oglašavanje novih dvorana

Atribut	Tip podatka	Opis varijable
id	BIGINT (Primarni ključ)	Jedinstveni identifikator oglasa dvorane
naz_dvorana	VARCHAR(255)	Naziv dvorane koja se želi oglašiti
kapacitet	INT	Kapacitet dvorane
opis	VARCHAR(1000)	Opis dvorane
termini_tjedan	VARCHAR(70)	Podatci o terminima po tjednu
ulica	VARCHAR(255)	Ulica u kojoj se nalazi dvorana
kucni_broj	VARCHAR(255)	Kućni broj na kojem se nalazi dvorana

Atribut	Tip podatka	Opis varijable
kategorija	INT (Strani ključ)	Identifikator kategorije(namjene) dvorane
user_id	INT (Strani ključ)	Identifikator korisnika(iznajmljivača) koji oglašava dvoranu
mjesto	INT (Streni ključ)	Identifikator mjesta u kojem se nalzi dvorana

Slika_dvorana

Sadrži sve slike dvorana

Atribut	Tip podatka	Opis varijable
id_slika	BIGINT (Primarni ključ)	Jedinstveni identifikator slike
image_data	BINARY ARRAY	Binary lista koja sprema sliku
id_dvorana	BIGINT (Strani ključ)	Identifikator dvorane kojoj pripada slika

Dijagram baze podataka

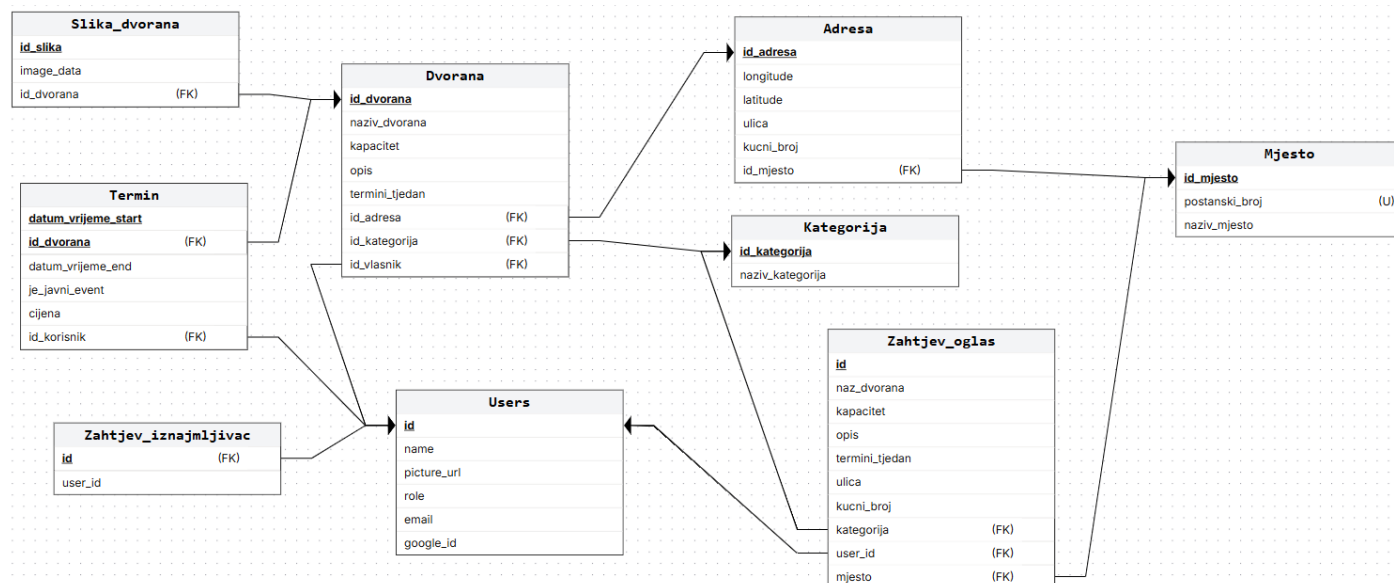


Figure 3: Slika

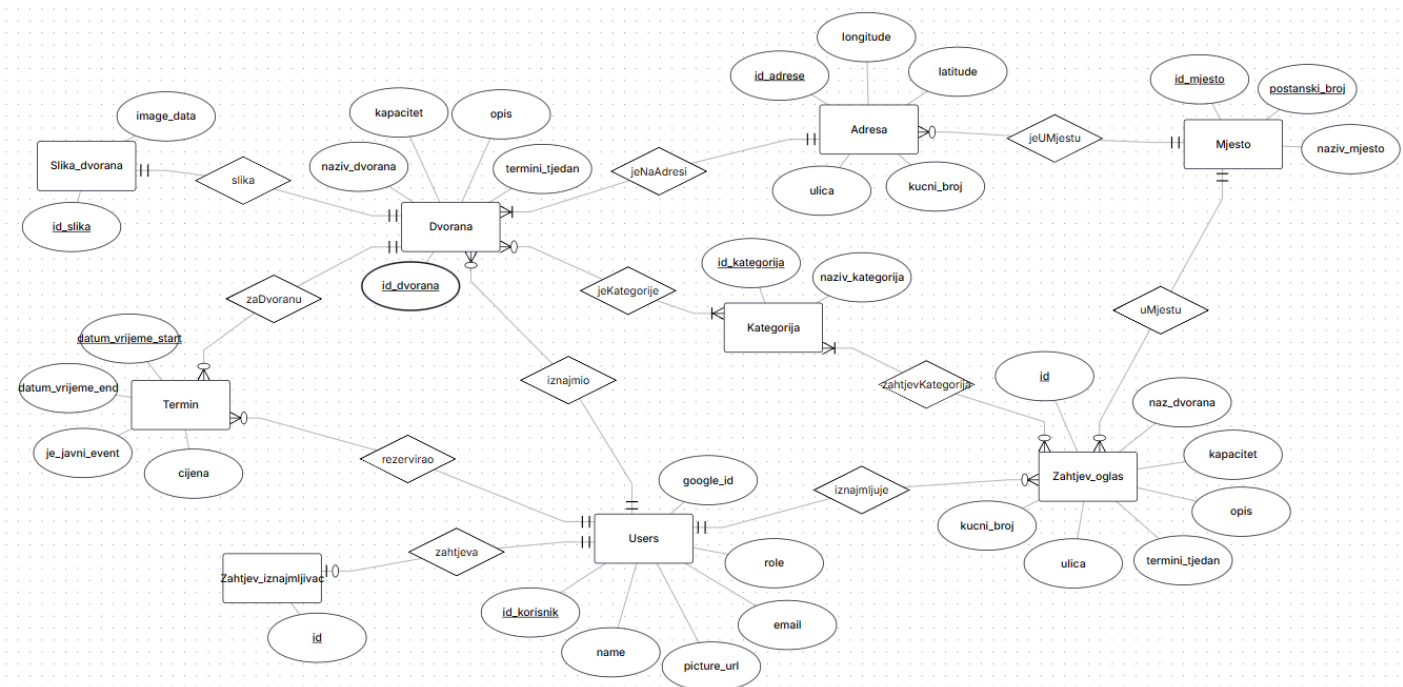


Figure 4: Slika

Dijagram razreda

Cijela Web-Aplikacija



Aplikacija je strukturirana po slojevitoj Spring Boot arhitekturi i sastoji se od više slojeva. Glavni slojevi su controller, service, repository, model, dto, security, utils, dok glavna klasa DvorankoAPIApplication služi kao ulazna točka aplikacije.

Controller sloj sadrži REST kontrolere koji primaju HTTP zahtjeve s frontenda i proslijeđuju ih servisima na obradu. Svaki kontroler odgovara određenom dijelu poslovne logike, primjerice UserController, DvoranaController ili KategorijaController. Kontroleri koriste Spring anotacije poput @RestController, @RequestMapping, @GetMapping i @PostMapping kako bi definirali API rute. Kada stigne zahtjev, kontroler ga proslijeđuje odgovarajućem servisu. Service sloj sadrži izvršnu logiku aplikacije. Servisi implementiraju glavne funkcionalnosti, kao što su kreiranje korisnika, dohvat dvorana ili obrada zahtjeva, te koriste repository sloj za pristup bazi podataka.

Repository sloj sadrži interfejs koji nasljeđuju JpaRepository i time omogućuju automatske CRUD operacije nad modelima. Spring automatski implementira ove interfejs u pozadini, čime se pojednostavljuje pristup podacima u bazi.

Model sloj sadrži JPA entitete koji predstavljaju tablice u bazi podataka. Svaka klasa je anotirana s @Entity i sadrži polja koja odgovaraju kolonama u tablici. Entiteti su međusobno povezani pomoću relacija kao što su @OneToMany, @ManyToOne i @ManyToMany. Među modelima se nalaze User, Role, Dvorana, Mjesto, Kategorija, SlikaDvorana, ZahtevOglas i ZahtevIznajmljivac, koji zajedno čine osnovu podatkovnog sloja aplikacije.

DTO sloj sadrži objekte za prijenos podataka između slojeva, čime se sprječava izravno izlaganje JPA entiteta prema klijentima. DTO objekti se koriste prilikom primanja podataka iz zahtjeva i slanja odgovora prema frontendu. U podpaketu createRequest nalaze se DTO klase koje služe za kreiranje novih entiteta, poput UserDTO, DvoranaDTO, KategorijaDTO i drugih. Kontroleri primaju DTO objekte, proslijeđuju ih servisima, a servisi ih zatim mapiraju na odgovarajuće modele.

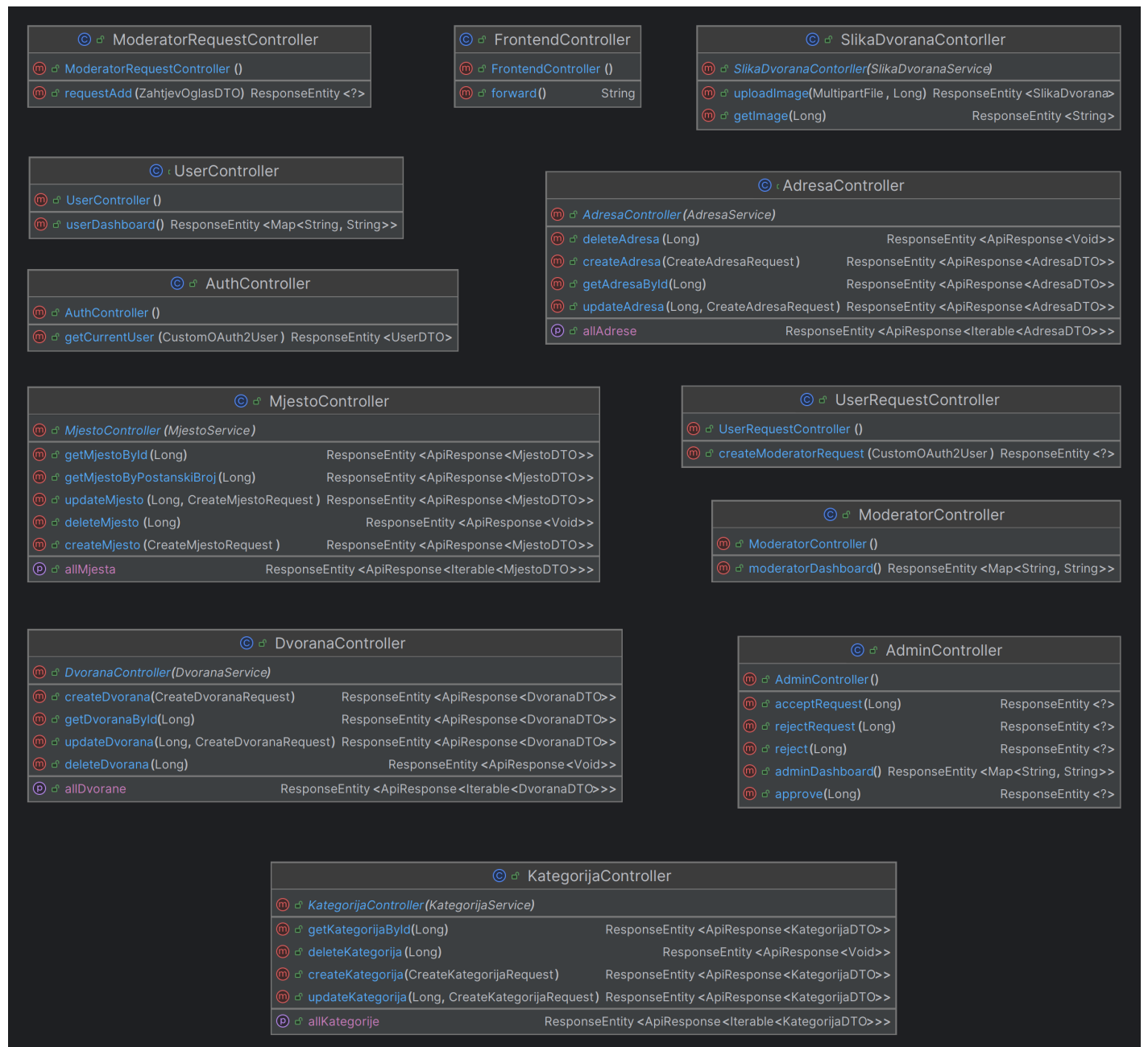
Security sloj implementira autentifikaciju i autorizaciju pomoću OAuth2 protokola. Klasa CustomOAuth2User predstavlja prilagođeni korisnički model koji dohvaća podatke o korisniku od OAuth providera. SecurityConfig konfigurira sigurnosna pravila, definira koje rute zahtijevaju prijavu i upravlja procesom prijave i odjave korisnika.

Utils sloj sadrži pomoćne klase koje se koriste u različitim dijelovima projekta. DtoMapper služi za konverziju između modela i DTO objekata, dok ApiResponse predstavlja generičku strukturu za vraćanje standardiziranih odgovora iz API-ja, uključujući status, poruku i podatke. Glavna klasa DvorankoAPIApplication anotirana je s @SpringBootApplication i služi kao ulazna točka aplikacije, pokreće Spring aplikacija i inicijalizira sve beanove potrebne za rad sustava.

Modeli

4.2 slika - Dijagram razreda modela

Kontroleri



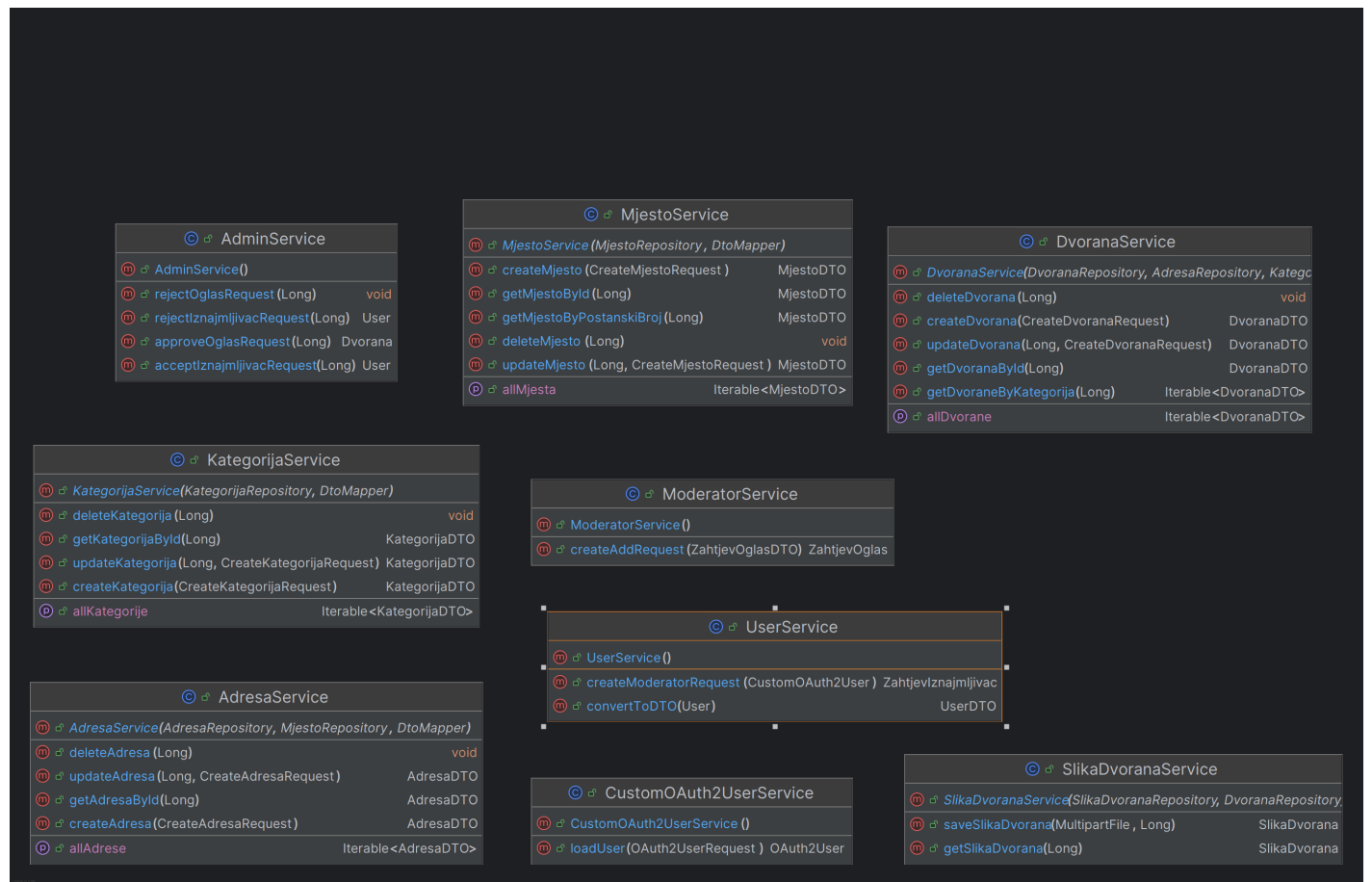
4.3 slika - Dijagram rezreda kontrolera

DTO

🔗 DvoranaDTO 🔗 DvoranaDTO(Long, String, Integer, String, AdresaDTO, Iterable<KategorijaDTO>) 🔗 DvoranaDTO() 🔗 kapacitetInteger 🔗 vlasnikUserDTO 🔗 kategorijeIterable<KategorijaDTO> 🔗 idDvoranaLong 🔗 nazivDvoranaString 🔗 slikaSlikaDvoranaDTO 🔗 opisString 🔗 adresaAdresaDTO 🔗 toString()String 🔗 canEqual(Object)boolean 🔗 hashCode()int 🔗 equals(Object)boolean 🔗 slikaSlikaDvoranaDTO 🔗 vlasnikUserDTO 🔗 kapacitetInteger 🔗 idDvoranaLong 🔗 adresaAdresaDTO 🔗 kategorijeIterable<KategorijaDTO> 🔗 opisString 🔗 nazivDvoranaString	📦 package fer.leprogi.dvoranko.dto.createRequest 🔗 MjestoDTO 🔗 MjestoDTO() 🔗 MjestoDTO(Long, Long, String) 🔗 nazivMjestoString 🔗 idMjestoLong 🔗 postanskiBrojLong 🔗 toString()String 🔗 equals(Object)boolean 🔗 canEqual(Object)boolean 🔗 hashCode()int 🔗 nazivMjestoString 🔗 idMjestoLong 🔗 postanskiBrojLong	🔗 UserDTO 🔗 UserDTO(Long, String, String, String, Role) 🔗 UserDTO() 🔗 pictureUrlString 🔗 idLong 🔗 roleRole 🔗 emailString 🔗 nameString 🔗 hashCode()int 🔗 canEqual(Object)boolean 🔗 toString()String 🔗 equals(Object)boolean 🔗 pictureUrlString 🔗 nameString 🔗 roleRole 🔗 idLong 🔗 emailString
🔗 ZahtjevOglasDTO 🔗 ZahtjevOglasDTO(Long, UserDTO, String, String, Integer, String) 🔗 ZahtjevOglasDTO() 🔗 nazivString 🔗 opisString 🔗 adresaString 🔗 latitudeDouble 🔗 idLong 🔗 userUserDTO 🔗 longitudeDouble 🔗 kapacitetInteger 🔗 kategorijeIterable<KategorijaDTO> 🔗 equals(Object)boolean 🔗 toString()String 🔗 canEqual(Object)boolean 🔗 hashCode()int 🔗 userUserDTO 🔗 nazivString 🔗 latitudeDouble 🔗 kapacitetInteger 🔗 idLong 🔗 kategorijeIterable<KategorijaDTO> 🔗 adresaString 🔗 opisString 🔗 longitudeDouble	🔗 KategorijaDTO 🔗 KategorijaDTO(Long, String) 🔗 KategorijaDTO() 🔗 idKategorijaLong 🔗 nazivKategorijeString 🔗 equals(Object)boolean 🔗 canEqual(Object)boolean 🔗 hashCode()int 🔗 toString()String 🔗 nazivKategorijeString 🔗 idKategorijaLong	🔗 SlikaDvoranaDTO 🔗 SlikaDvoranaDTO() 🔗 idSlikaLong 🔗 imageDatabyte[] 🔗 equals(Object)boolean 🔗 canEqual(Object)boolean 🔗 hashCode()int 🔗 toString()String 🔗 idSlikaLong 🔗 imageDatabyte[]
	🔗 ZahtjevIznajmljivacDTO 🔗 ZahtjevIznajmljivacDTO(Long, UserDTO) 🔗 ZahtjevIznajmljivacDTO() 🔗 idLong 🔗 userUserDTO 🔗 equals(Object)boolean 🔗 canEqual(Object)boolean 🔗 hashCode()int 🔗 toString()String 🔗 idLong 🔗 userUserDTO	🔗 AdresaDTO 🔗 AdresaDTO(Long, Double, Double, String, String, MjestoDTO) 🔗 AdresaDTO() 🔗 ulicaString 🔗 longitudeDouble 🔗 kucniBrojString 🔗 latitudeDouble 🔗 mjestoMjestoDTO 🔗 idAdresaLong 🔗 hashCode()int 🔗 equals(Object)boolean 🔗 canEqual(Object)boolean 🔗 toString()String 🔗 mjestoMjestoDTO 🔗 kucniBrojString 🔗 latitudeDouble 🔗 ulicaString 🔗 idAdresaLong 🔗 longitudeDouble

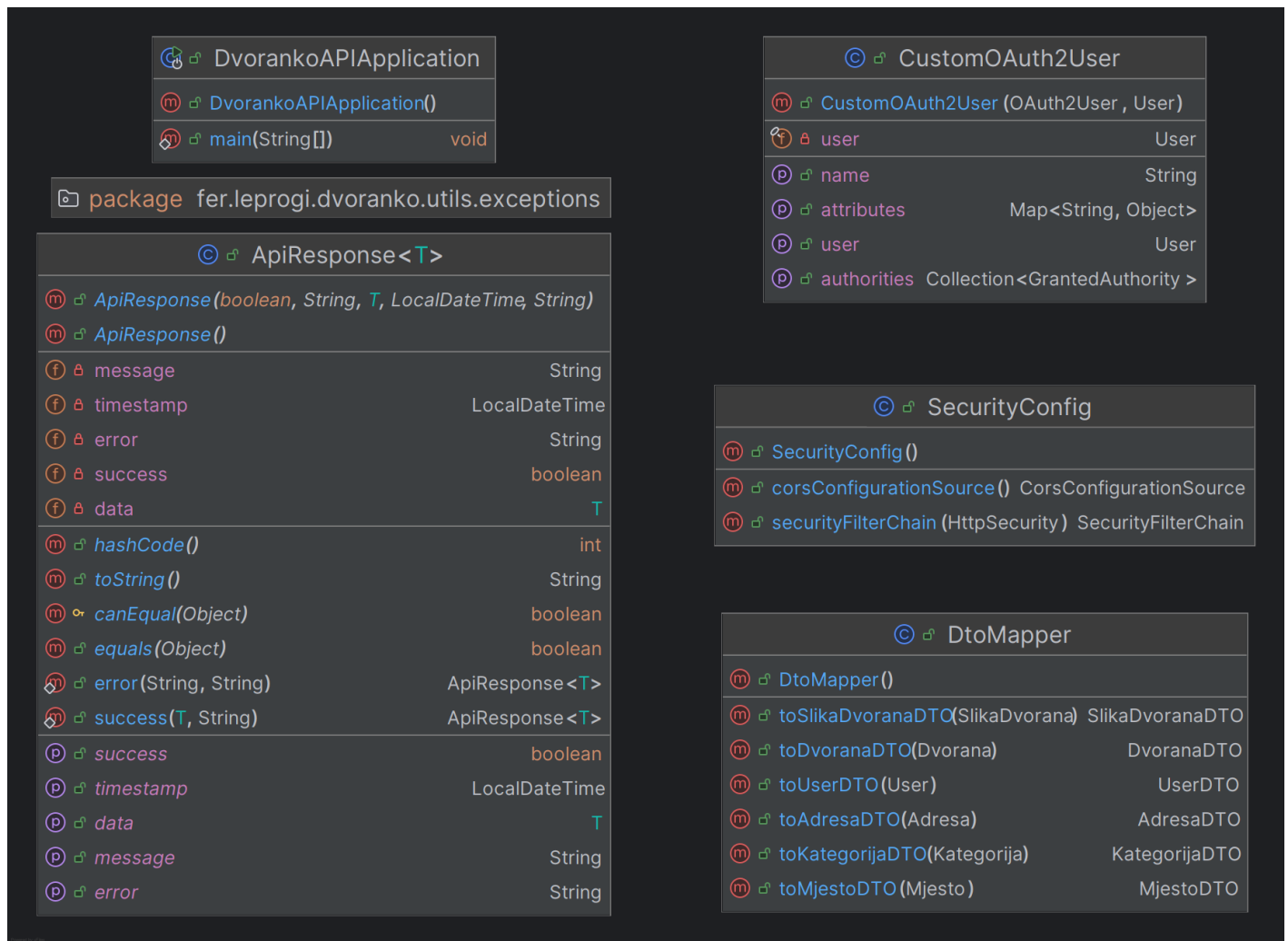
4.4 slika - Dijagram razreda DTO-a

Servisi



4.5 slika - Dijagram razreda servisi

Sigurnost i pomoćne klase



4.6 slika - Dijagram razreda za security i utils

Dinamičko ponašanje aplikacije

Dinamičko ponašanje aplikacije odnosi se na način na koji objekti u sustavu evoluiraju kroz vrijeme, uključujući prijelaze između različitih stanja. To uključuje aktivnosti, događaje, odluke i interakcije unutar aplikacije. UML dijagrami stanja omogućuju vizualizaciju tih promjena i olakšavaju razumijevanje dinamike sustava.

Razumijevanje promjena stanja neophodno je za pravilno funkcioniranje aplikacije jer pruža uvid u interakcije među objektima, komponentama i korisnicima tijekom rada sustava. Korištenjem UML dijagrama stanja i aktivnosti moguće je vizualizirati prijelaze i stanja objekata, identificirati potencijalne probleme, osigurati točnu implementaciju te poboljšati komunikaciju među članovima tima.

Zadatak:

- Dokumentirajte dva dinamička dijagrama koji prikazuju značajne ili neke složene procese u aplikaciji. Registracija i prijava korisnika nisu ključni procesi u ovom kontekstu, stoga se usmjerite na specifične, značajne procese.

UML dijagrami stanja

UML dijagrami stanja nužni su za razumijevanje dinamičkog ponašanja sustava. Oni jasno prikazuju promjene stanja objekata tijekom vremena ovisno o događajima i uvjetima.

Preporuke za izradu UML dijagrama stanja:

- Odaberite značajan objekt sustava (npr. prijava štete, obrada zahtjeva).

2. Identificirajte stanja objekta (npr. aktivan, neaktivan, obrisano).
3. Definirajte događaje i uvjete prijelaza.
4. Ako je potrebno, koristite hijerarhiju stanja za složenije probleme.
5. Dodajte bilješke za pojašnjenje važnih prijelaza i elemenata.

Primjer: Dijagram stanja za prijavu štete može uključivati:

- Stanja: *Kreirana prijava*, *Obradena prijava*, *Zatvorena prijava*...

UML dijagrami aktivnosti

Dijagram aktivnosti prikazuje tijek izvršavanja određenog procesa. Osim za razumijevanje toka podataka unutar aplikacije, koristi se za analizu poslovnih procesa.

Zadatak:

1. Identificirajte proces koji želite modelirati (npr. obrada zahtjeva).
2. Razlomite proces na aktivnosti i povežite ih za prikaz slijeda izvršavanja.
3. Upotrijebite pseudo čvorove za jasan prikaz.

Primjer:** Na primjer, u aplikaciji za upravljanje štetama, dijagram aktivnosti može prikazivati proces obrade zahtjeva za pomoć, što uključuje aktivnosti: *Primanje zahtjeva*, *Analiza podataka*, *Odobranje resursa*, *Slanje pomoći*.

Arhitektura sustava predstavlja temeljni okvir za razumijevanje i implementaciju svih njegovih funkcionalnosti. U kontekstu razvojne dokumentacije aplikacija, dijagrami komponenata i razmještaja odlučujući su za prikaz povezanosti i rasporeda različitih komponenata sustava. Ovi dijagrami omogućuju sudionicima projekta razumijevanje i vizualizaciju fizičkog i logičkog dizajna sustava, uključujući interakcije između dijelova aplikacije, što je odlučujuće za efikasnu implementaciju i dugoročnu održivost sustava.

Arhitektura sustava, u kontekstu dijagrama komponenata i razmještaja, pruža uvid u strukturu i raspored ključnih dijelova aplikacije. Ovi dijagrami nisu korisni samo tijekom faza oblikovanja i implementacije, već služe i kao alati za održavanje i optimizaciju sustava u budućnosti.

Kao dio razvoja aplikacije, važno je osmisлити i dokumentirati arhitekturu sustava s naglaskom na dijagrame komponenata i razmještaja. Vaš zadatak je izraditi **dijagram komponenata** koji će jasno prikazivati ključne funkcionalne komponente aplikacije, njihovu međusobnu povezanost te sučelja za komunikaciju. Također, trebate izraditi **dijagram razmještaja** komponenata, koji treba detaljno prikazivati kako su te komponente raspoređene u infrastrukturi sustava, uključujući fizičke i virtualne resurse poput poslužitelja ili uređaja krajnjih korisnika.

Dijagram komponenata

Komponente sustava predstavljaju bitne dijelove aplikacije koji obavljaju specifične funkcije. Svaka komponenta je autonomna jedinica s vlastitim odgovornostima, ali je povezana s drugim komponentama kako bi sustav u cjelini funkcionirao. Komponente mogu biti elementi poput modula, servisa, razreda ili paketa, te komuniciraju putem jasno definiranih sučelja.

UML dijagram komponenata za vašu aplikaciju ovisi o njezinoj arhitekturi i složenosti, no općenito treba prikazivati važne funkcionalne komponente, njihovu međusobnu povezanost i sučelja za komunikaciju. Obzirom na namjenu projekta, dijagram treba biti jasan, organiziran i lako čitljiv kako bi olakšao razumijevanje strukture i suradnju unutar tima.

Dijagram razmještaja

UML dijagram razmještaja prikazuje fizičku ili virtualnu raspodjelu komponenata sustava unutar infrastrukture. Cilj je prikazati kako su komponente raspoređene (npr. na poslužiteljima, u okruženjima oblaka ili na uređajima krajnjih korisnika) te način komuni čije, API-ja ili drugih komunikacijskih protokola.

U ovoj dokumentaciji preporučuje se uključiti dijagram razmještaja instanci (engl. Instance Level Deployment Diagram) ili implementacije.

- Dijagram razmještaja instanci prikazuje način na koji su aplikacijske komponente raspoređene unutar infrastrukture, uključujući fizičke i virtualne čvorove (poslužitelje) na kojima se izvode. Ovaj dijagram detaljno opisuje kako su komponente povezane i kako međusobno komuniciraju.

U kontekstu aplikacije, npr. ona koja koristi Docker kontejner, dijagram razmještaja instanci pokazuje kako se aplikacije raspoređuju unutar Docker kontejnera na različitim poslužiteljima ili u okruženjima oblaka.

Implementacijski oblik

- Implementacijski oblik daje detaljan prikaz rasporeda komponenata u fizičkoj ili virtualnoj infrastrukturi. Ovdje se koriste stvarni poslužitelji, mrežne veze, uređaji korisnika, aplikacijski paketi i drugi artefakti kako bi dijagram prikazao točan fizički raspored komponenata, s naglaskom na fizičko povezivanje i tehničke resurse.

Primjeri: Raspored sustava unutar fizičkog podatkovnog centra, uključujući informacije o virtualnim poslužiteljima, bazama podataka, mrežnim vezama i sklopovskim resursima; prikaz rasporeda komponenata na konkretnim poslužiteljima u oblaku (npr. AWS, Google Cloud).

Ovo poglavlje treba opisati provedena ispitivanja implementiranih funkcionalnosti na razini komponenti i sustava. Fokus je na odabiru i izvedbi ispitnih slučajeva koji obuhvaćaju redovne, rubne uvjete i testiranje grešaka, kao i upotrebu odgovarajućih alata za provedbu testiranja.

Ispitivanje komponenti

Cilj ispitivanja komponenti je provjera osnovnih funkcionalnosti implementiranih u razredima sustava. Ovdje je potrebno izolirati svaku komponentu kako bi se testirala njezina ispravnost i reakcija na različite scenarije.

Zadaci:

1. Razviti minimalno **6 ispitnih slučajeva** koji obuhvaćaju:
 - **Redovne slučajeve:** testiranje uobičajenog ponašanja funkcionalnosti.
 - **Rubne uvjete:** provjera ulaznih podataka na granici valjanosti.
 - **Izazivanje pogreške (exception throwing):** testiranje reakcije na iznimke.
 - **Nepostojeće funkcionalnosti:** provjera reakcije na poziv neimplementirane funkcionalnosti.

Struktura ispitivanja:

Za svaki ispitni slučaj potrebno je:

1. Opišite funkcionalnost koju testirate (npr. dodavanje korisnika, validacija podataka).
2. Navedite ispitni slučaj:
 - Ulazne podatke.
 - Očekivane rezultate.
 - Dobivene rezultate (prolaz/pad ispitivanja).
3. Opišite postupak provođenja ispitivanja
4. U Gitu moraju biti dostupni izvorni kodovi ispitnih slučajeva.

Ispitivanje sustava

Cilj ispitivanja sustava je testiranje ponašanja cijelog sustava u uvjetima stvarnog korištenja, uz posebnu pažnju na međusobnu povezanost svih komponenti. Ispitivanje treba obuhvatiti sve aspekte sustava i njegovu interakciju s korisnicima.

Zadaci:

Razviti minimalno **4 ispitna slučaja** koji obuhvaćaju:

- **Redovne slučajeve:** očekivano ponašanje sustava.
- **Rubne uvjete:** reakcija sustava na granične ulaze.
- **Poziv nepostojećih funkcionalnosti:** testiranje kako sustav reagira na neimplementirane ili neispravne funkcije.

Struktura ispitnih slučajeva za Selenium:

1. **Ulazi:**

- Konkretni podaci koji se unose u sustav (npr. korisničko ime i lozinka za prijavu).
- Simulacija korisničkih akcija (klikanje, unos teksta, navigacija).

2. Koraci ispitivanja:

- Npr. Detaljan opis koraka koje Selenium izvršava:
 1. Otvoriti aplikaciju u pregledniku.
 2. Unijeti podatke u formu.
 3. Kliknuti na gumb za potvrdu.
 4. Verificirati očekivane rezultate (npr. prijava uspješna).

3. Očekivani izlaz:

- Očekivani rezultat ispitivanja (npr. korisnik preusmjeren na početnu stranicu nakon prijave).

4. Dobiveni izlaz:

- Priložiti logove, screenshotove ili izvještaje s generiranim rezultatima (npr. iz JUnit-a ili Selenija).

Alati za ispitivanje sustava:

- **Selenium IDE** ili prikladan obzirom na vaše razvojno okruženje:
 - Jednostavan alat za snimanje korisničkih akcija u pregledniku i automatsko ponavljanje testova.
 - Preporučuje se za osnovne ispitne slučajeve.
- **Selenium WebDriver:**
 - Omogućuje pisanje naprednih testova u različitim programskim jezicima (Java, Python, C#).
 - Preporučuje se za složenije testove, koji zahtijevaju detaljno prilagodbu i automatizaciju.

Primjeri ispitivanja Seleniomom:

1. Formular za prijavu: > u dokumentaciji obavezna su specifičnija ispitivanja vaše aplikacije!

- **Ulaz:** Korisničko ime = “user@fer.ugnz.hr”, Lozinka = “password123”.
- **Koraci:**
 1. Otvoriti aplikaciju.
 2. Unijeti korisničko ime i lozinku.
 3. Kliknuti na “Prijava”.
 4. Provjeriti je li korisnik preusmjeren na početnu stranicu.

- **Očekivani izlaz:** “Prijava uspješna.”

2. Rubni uvjet – nevažeća lozinka:

- **Ulaz:** Korisničko ime = “user@example.com”, Lozinka = “malimedo”.
- **Koraci:**
 1. Unijeti podatke u formu za prijavu.
 2. Kliknuti na “Prijava”.
 3. Provjeriti je li prikazana poruka o grešci.
- **Očekivani izlaz:** “Pogrešno korisničko ime ili lozinka.”

Prezentacija rezultata

Za oba tipa ispitivanja (komponenti i sustava) potrebno je:

- Jasno dokumentirati ulaze, korake, očekivane i dobivene rezultate.
- Priložiti slike ekrana, logove ili izvješća generirana alatima (npr. JUnit ili Selenium).

- Detaljno opisati ponašanje sustava, posebno u rubnim uvjetima.
- Navesti broj otkrivenih grešaka!

Korištene tehnologije i alati

Cilj: Jasno i precizno opisati tehnologije korištene u projektu kako bi se olakšalo održavanje, proširenje i suradnja u timu. Uključite informacije:

1. **Programski jezici:** Navesti korištene jezike i njihove verzije (npr. JavaScript 16.13).
2. **Radni okviri i biblioteke:** Detaljno opisati alate za frontend i backend (npr. React 18, Node.js 16).
3. **Baza podataka:** Navesti vrstu baze (npr. PostgreSQL 13).
4. **Razvojni alati:** Popis korištenih IDE-ova, alata za verzioniranje (npr. VS Code, Git 2.34).
5. **Alati za ispitivanje:** Jedinični, integracijski ili UI ispitni sljučajevi (npr. Jest 27, Selenium 4.0).
6. **Alati za razmjestaj:** Korišteni alati za implementaciju (npr. Docker 20.10).
7. **Cloud platforma:** Ako je aplikacija hostana, navesti platformu (npr. AWS).

Preporuke za opis:

- **Jasno i precizno:** Izbjegavati tehnički žargon i navesti točne verzije.
- **Obrazloženje izbora:** Objasniti zašto su odabrane određene tehnologije.
- **Opis konfiguracije:** Istaknuti specifične postavke alata i baza.

Primjer:

Za razvoj klijentskog dijela aplikacije korišten je **React** (verzija 18) [ref.], popularna JavaScript biblioteka za izgradnju interaktivnih korisničkih sučelja. React omogućuje stvaranje samostalnih komponenti koje se mogu ponovno koristiti i lako ažurirati, čime se poboljšava učinkovitost razvoja. Za stiliziranje su upotrijebljene **styled-components** (verzija 5.3), što omogućava integraciju stilova izravno unutar Ručat komponenti koristeći **CSS-in-JS** pristup.

Ovaj odjeljak dokumentacije treba dati detaljne smjernice za instalaciju, konfiguraciju, pokretanje i administraciju aplikacije. Cilj je olakšati postavljanje aplikacije na razvojnom, ispitnom i produkcijskom okruženju.

1. Instalacija

Ovdje treba navesti korake potrebne za instalaciju svih potrebnih komponenti:

- **Preduvjeti:** Popis potrebnog softvera i njihovih verzija (npr. Node.js 16, Docker 20.10).
- **Preuzimanje:** Upute za preuzimanje izvornog koda (npr. kloniranje Git repozitorija).

Primjer:

```
git clone https://github.com/Projekt/primjer.git
cd repo
```

Instalacija ovisnosti: Upute za instaliranje ovisnosti.

```
npm install
```

2. Postavke

Detaljne upute za konfiguraciju aplikacije:

- **Konfiguracijske datoteke:** Gdje se nalaze (npr. config.json, .env) i što treba prilagoditi.

– Primjer .env datoteke:

```
DATABASE_URL=postgres://user:password@localhost:5432/dbname
API_KEY=your_api_key
```

- **Postavke baze podataka:** Upute za inicijalizaciju baze podataka, uključujući migracije i postavljanje inicijalnih podataka.

```
npm run db:migrate
```

```
npm run db:seed
```

3. Pokretanje aplikacije

Upute za pokretanje aplikacije u različitim okruženjima:

- **Razvojno okruženje:**

```
npm run dev
```

- **Produksijsko okruženje:**

- Prevođenje aplikacije:

```
npm run build
```

- Pokretanje poslužitelja:

```
npm start
```

- **Provjera rada:** Navesti npr. URL (npr. `http://localhost:3000`) .

4. Upute za administratore

Smjernice za administratore aplikacije nakon puštanja u pogon:

- **Pristup administratorskom sučelju:**

- URL za admin panel (npr. `/admin`).

- Početni podaci za prijavu (ako postoje).

- **Redovito održavanje:**

- Arhiviranje baze podataka.

- Pregled logova.

- Ažuriranje aplikacije (primjer: povlačenje novih verzija iz Git repozitorija i ponovno pokretanje aplikacije).

```
git pull origin main
```

```
npm install
```

```
npm run build
```

```
npm start
```

- **Rješavanje problema:** Kako pristupiti logovima i dijagnosticirati greške (npr. `logs/error.log` ili `docker logs`).

5. Primjer za Render platformu (Cloud Deploy)

Render je popularna cloud platforma za jednostavno smještanje aplikacija.

- **Priprema repozitorija:**

- Osigurajte da vaš projekt ima datoteku `render.yaml` ili `Dockerfile` za konfiguraciju deploja.

- Primjer `render.yaml`:

```
services:
```

```
- type: web
```

```
name: my-web-app
```

```
env: node
```

```
buildCommand: npm install && npm run build
```

```
startCommand: npm start
```

```
plan: free
```

- **Postavljanje na Render:**

- Prijavite se na Render.
- Kreirajte novi **Web Service** i povežite ga s vašim GitHub repozitorijem.
- Konfigurirajte postavke (npr. build i start komande).
- Dodajte environment varijable (npr. DATABASE_URL, API_KEY).

- **Pokretanje aplikacije:**

Render će automatski preuzeti repozitorij, instalirati ovisnosti i pokrenuti aplikaciju. Nakon deploya, aplikaciji možete pristupiti putem generiranog URL-a (npr. <https://my-web-app.onrender.com>).

Opis prisutpa aplikaciji na javnom poslužitelju

Pristup aplikaciji Dokumentirajte postupak i pružite jasne smjernice za korištenje aplikacije na javnom poslužitelju. - Navedite ograničenja!

- U uputama obuhvatite kako korisnici mogu pristupiti aplikaciji putem internetskog preglednika.
- Priložite korake za pristup administratorskom sučelju ako je primjenjivo.

U ovom poglavlju potrebno je napisati osvrt na vrijeme izrade projektnog zadatka, koji su tehnički izazovi prepoznati, jesu li riješeni ili kako bi mogli biti riješeni, koja su znanja stečena pri izradi projekta, koja bi znanja bila posebno potrebna za brže i kvalitetnije ostvarenje projekta i koje bi bile perspektive za nastavak rada u projektnoj grupi. Potrebno je točno popisati funkcionalnosti koje nisu implementirane u ostvarenoj aplikaciji.

Kontinuirano osvježavanje Popisati sve reference i literaturu koja je pomogla pri ostvarivanju projekta. 1. Programsko inženjerstvo, FER ZEMRIS, <http://www.fer.hr/predmet/proinz> 2. I. Sommerville, "Software engineering", 8th ed, Addison Wesley, 2007. 3. T.C.Lethbridge, R.Langaniere, "Object-Oriented Software Engineering", 2nd ed. McGraw-Hill, 2005. 4. I. Marsic, Software engineering book, Department of Electrical and Computer Engineering, Rutgers University, <http://www.ece.rutgers.edu/~marsic/books/SE> 5. The Unified Modeling Language, <https://www.uml-diagrams.org/> 6. Astah Community, <http://astah.net/editions/uml-new>

Rev.	Opis promjene	Autor	Datum
0.1.	Napravljen predložak za wiki	D.Maras	13.10.2025.
0.2.	Informacije o timu	E.Huljaj	13.10.2025.
1.1.	Početni opis projektnog zadatka	E.Huljaj	23.10.2025.
2.1.	Raspisani funkcionalni i nefunkcionalni zahtjevi	E.Huljaj	25.10.2025.
4.1.	Opis tablica baze podataka	I.Vučak	29.10.2025.
4.2.	Opis arhitekture sustava	I.Vučak	29.10.2025.
4.3.	Dijagram baze podataka	I.Vučak	29.10.2025.
2.2.	Opis dionika sustava	E.Huljaj	1.11.2025.
4.4.	Nadopunjen opis baze podataka (popravljen tablice i dijagrami)	I.Vučak	3.11.2025.
3.1.	Dijagrami obrazaca uporabe	D.Maras	3.11.2025.
3.2.	Interakcije glavnih korisničkih aktera sa sustavom	D.Maras	3.11.2025.
3.3.	Opis obrazaca uporabe	D.Maras	3.11.2025.
3.4.	Ispravak obrazaca uporabe	E.Huljaj	10.11.2025.
2.3.	Ispravak funkcionalnih zahtjeva prema obrazcima uporabe	E.Huljaj	10.11.2025.
3.5.	Provjera uključenosti funkcionalnosti u obrasce uporabe	E.Huljaj	10.11.2025.
1.2.	Ispravak opisa projektnog zadatka prema novim F/NF zahtjevima	E.Huljaj	11.11.2025.
1.3.	Napisan opseg projektnog zadatka	E.Huljaj	11.11.2025.
4.5.	Opisani frontend i backend slojevi aplikacije, popravljen opis arhitekture	M.Vandek	12.11.2025.
3.6.	Sekvencijski dijagrami	D.Maras	12.11.2025.
4.6.	Popravljen nazivlje u tablicama baze podataka	I.Vučak	13.11.2025.
2.4.	Ispravak popisa dionika	V.M.Lulić	13.11.2025.

Rev.	Opis promjene	Autor	Datum
------	---------------	-------	-------

*Rev. -> (broj strane na wiki).(broj promjene)

TO DO

Dnevnik sastajanja

U ovom dijelu potrebno je redovito osvježavati dnevnik sastajanja/tjedni izvještaj aktivnosti prema predlošku. Cilj predstaviti kratki opis ključnih sastanaka, **donesenih odluka** i zaduženja s tjednih sastanaka.

1. sastanak

- Datum: 9. listopada 2025.
- Prisustvovali: V.M.Lulić, B.Navratil, M.Vandek, D.Maras, I.Vučak, E.Huljaj
- Teme sastanka: > * uspostavljena Discord grupa sa svim članovima tima > * odabran naziv tima > * diskusija o vlastitoj ideji kao temi projekta te sastavljanje prijedloga > * inicijalna i okvirna podjela članova tima po ulogama

2. sastanak

- Datum: 16. listopada 2025.
- Prisustvovali: V.M.Lulić, B.Navratil, I.Vučak, E.Huljaj
- Teme sastanka: > * određeni funkcionalni i nefunkcionalni zahtjevi > * daljnja razrada funkcionalnosti aplikacije > * razrada strukture baze podataka

3. sastanak

- Datum: 3. studenoga 2025.
- Prisustvovali: V.M.Lulić, B.Navratil, I.Vučak, E.Huljaj, D.Maras, M.Vandek
- Teme sastanka: > * definirani obrasci uporabe > * definiran način implementacije Google Maps API funkcionalnosti > * određene potrebne promjene u izgledu web-aplikacije » * kako pristupamo login stranici » * popis dvorana prebaciti na početnu stranicu

Plan rada

Tablični/Gantt/Kanban prikaz * Prikaz vremenskog plana rada ključnih aktivnosti (tjedna granulacija) * Uključuje akronime angažiranih članova tima TODO: primjer

Tablica aktivnosti

Kontinuirano osvježavanje

Komponente	Vid Martin Lulić	Eva Huljaj	Dino Maras	Ivan Vučak	Matija Vandek	Borna Navratil
Upravljanje projektom	5					
Opis projektnog zadatka		6				
Funkcionalni zahtjevi	1	4.5		1.5	1	2
Opis pojedinih obrazaca		1	5			
Dijagram obrazaca			2.5			
Sekvencijski dijagrami			3.5			
Opis ostalih zahtjeva		1				
Arhitektura i dizajn sustava	2			5		
Izrada baze podataka				5	3	
Spajanje backenda s bazom podataka	3					
Dijagram razreda			2			
Dijagram stanja						
Dijagram aktivnosti						
Dijagram komponenti						

Komponente	Vid Martin Lulić	Eva Huljaj	Dino Maras	Ivan Vučak	Matija Vandek	Borna Navratil
Učenje tehnologija i alata	15					
Ispitivanje programskog rješenja						
Dijagram razmještaja						
Upute za puštanje u pogon						
Sastanci	5	5	5	5	5	5
Zaključak i budući rad						
Popis literature						
Uređivanje dokumentacije			1			
Izrada korisničkog sučelja	1	20				10
Izrada programske potpore	18		9		2	
Izrada prezentacije						
Dizajn korisničkog sučelja		4				
Pregledavanje dokumentacije	1.5	1	1.5		1	
Puštanje aplikacije u pogon	5			17		

Dijagram pregleda promjena

Prenijeti dijagram pregleda promjena nad datotekama projekta. Potrebno je na kraju projekta generirane grafove s githuba prenijeti u ovo poglavlje dokumentacije. Dijagrami za vlastiti projekt se mogu preuzeti s github.com stranice, u izborniku Repository, pritiskom na stavku Contributors.

Ključni izazovi i rješenja

- Zaključno
- Opis izazova: Glavni izazovi tijekom projekta (npr. kašnjenje u razvoju, tehnički problemi).
- Rješenja: Način na koji su izazovi riješeni, kao i naučene lekcije koje su doprinijele napretku tima.

TG10.1 LeProgi